



TECHNISCHE UNIVERSITÄT  
CHEMNITZ

Department of Electrical Engineering and Information  
Technology

Chair of Measurement and Sensor Technology

# Project Documentation

„Project Lab Embedded Systems“

Group: 15

Members: Tuan Minh Le  
Gia Khanh Nguyen  
Minh Quang Vu  
Khoi Nguyen Nguyen

Project: STM32-Based Multisine EIS Platform for Equivalent-  
Circuit Parameter Estimation

Date: 2026 - 07 - 23

# Contents

|                                                                                           |           |
|-------------------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                                           | <b>3</b>  |
| <b>Member Responsibilities</b>                                                            | <b>3</b>  |
| <b>Introduction</b>                                                                       | <b>4</b>  |
| Motivation & Problem . . . . .                                                            | 4         |
| Report Outline . . . . .                                                                  | 5         |
| <b>1 Theoretical Background</b>                                                           | <b>5</b>  |
| 1.1 Fundamentals of Impedance Spectroscopy . . . . .                                      | 5         |
| 1.1.1 Material Characterization Capabilities of IS . . . . .                              | 6         |
| 1.2 Requirements for Consistent EIS Measurements . . . . .                                | 7         |
| 1.2.1 Linearity . . . . .                                                                 | 7         |
| 1.2.2 Stability (Time-Invariance) . . . . .                                               | 8         |
| 1.2.3 Causality . . . . .                                                                 | 8         |
| 1.3 Classification of Measuring Platforms . . . . .                                       | 8         |
| 1.4 Embedded Measurement . . . . .                                                        | 9         |
| 1.4.1 Comparison of Measurement Methods . . . . .                                         | 9         |
| 1.4.2 Digital vs. Analog Signal Processing for Impedance Extraction . . . . .             | 10        |
| 1.4.3 Overall Block Diagram of the I-V-Based Architecture . . . . .                       | 10        |
| 1.5 Excitation Signals . . . . .                                                          | 11        |
| 1.5.1 Available Excitation Signal Types . . . . .                                         | 12        |
| 1.5.2 Optimization of Multisine Signals . . . . .                                         | 12        |
| 1.6 Acquisition of Current and Voltage Signals and Synchronization . . . . .              | 14        |
| 1.7 AC Signal Analysis . . . . .                                                          | 15        |
| 1.7.1 Primary Processing Methods . . . . .                                                | 15        |
| 1.7.2 Hardware Acceleration . . . . .                                                     | 15        |
| 1.7.3 From AC Analysis to Impedance . . . . .                                             | 16        |
| 1.8 Feature Extraction . . . . .                                                          | 16        |
| 1.8.1 From AC Signal to EIS Features . . . . .                                            | 16        |
| 1.8.2 Why Preserve the Sequence Instead of Extracting Mean/Variance Statistics? . . . . . | 17        |
| 1.9 Machine Learning & Deep Learning Fundamentals . . . . .                               | 17        |
| 1.9.1 Classification Theory with Random Forest . . . . .                                  | 17        |
| 1.9.2 Regression Theory with 1D-CNN . . . . .                                             | 18        |
| 1.9.3 Limitations of Traditional Models . . . . .                                         | 19        |
| 1.10 Machine Learning Deployment . . . . .                                                | 19        |
| <b>2 System Design and Implementation</b>                                                 | <b>21</b> |
| 2.1 System Architecture Overview . . . . .                                                | 21        |
| 2.2 Hardware Setup . . . . .                                                              | 23        |
| 2.2.1 Circuit Design . . . . .                                                            | 23        |
| 2.2.2 Stable 1.65V Reference Supply . . . . .                                             | 25        |
| 2.2.3 The DAC Output Stage . . . . .                                                      | 26        |
| 2.2.4 The Device Under Test (DUT) . . . . .                                               | 27        |
| 2.2.5 The ADC Front-End (Signal Measurement) . . . . .                                    | 27        |
| 2.3 Firmware Implementation . . . . .                                                     | 29        |
| 2.4 Graphical User Interface for Real-Time EIS Visualization . . . . .                    | 31        |
| 2.4.1 Purpose and Role of the GUI . . . . .                                               | 31        |
| 2.4.2 Serial Communication and Data Acquisition . . . . .                                 | 32        |
| 2.4.3 Graphical Interface Design . . . . .                                                | 32        |

|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| 2.4.4    | Nyquist Plot . . . . .                                                             | 34        |
| 2.4.5    | Bode Plots . . . . .                                                               | 35        |
| 2.4.6    | Additional Visualization Modes . . . . .                                           | 36        |
| 2.4.7    | Use of the GUI for Debugging and Validation . . . . .                              | 36        |
| 2.4.8    | Data Logging and Relation to Machine Learning . . . . .                            | 37        |
| 2.5      | Data Acquisition and Dataset Preparation . . . . .                                 | 37        |
| 2.5.1    | Dataset Construction from Raw Data Streams . . . . .                               | 37        |
| 2.5.2    | Cleaning and Splitting Measurement Data Frames . . . . .                           | 38        |
| 2.5.3    | Flattening Frequency Sequence Data into Machine Learning Feature Vectors . . . . . | 38        |
| 2.5.4    | Dual Scaling Strategy . . . . .                                                    | 39        |
| 2.5.5    | Data Leakage Prevention . . . . .                                                  | 40        |
| 2.6      | Machine Learning Model Design . . . . .                                            | 40        |
| 2.6.1    | Two-Stage Cascade Architecture . . . . .                                           | 40        |
| 2.6.2    | Stage 1 — Topology Classifier (Random Forest) . . . . .                            | 40        |
| 2.6.3    | Stage 2 — Topology-Specific Parameter Regressors (1D-CNN) . . . . .                | 41        |
| 2.7      | Model Training and Deployment . . . . .                                            | 42        |
| 2.7.1    | Model Training Techniques . . . . .                                                | 42        |
| 2.7.2    | Embedded Deployment . . . . .                                                      | 43        |
| <b>3</b> | <b>Results and Evaluation</b>                                                      | <b>44</b> |
| 3.1      | Evaluation Methodology . . . . .                                                   | 44        |
| 3.2      | Impedance Measurement Evaluation . . . . .                                         | 45        |
| 3.3      | Machine Learning Results . . . . .                                                 | 46        |
| 3.4      | Embedded Deployment Results . . . . .                                              | 49        |
| 3.5      | Limitations . . . . .                                                              | 49        |
| 3.5.1    | IS Measurement System Limitations . . . . .                                        | 49        |
| 3.5.2    | Machine Learning Approach . . . . .                                                | 49        |
| <b>4</b> | <b>Conclusion</b>                                                                  | <b>50</b> |
| 4.1      | Summary . . . . .                                                                  | 50        |
| 4.2      | Main Results . . . . .                                                             | 51        |
| 4.3      | Limitations . . . . .                                                              | 51        |
| <b>5</b> | <b>References</b>                                                                  | <b>52</b> |
| <b>A</b> | <b>Complete RCE Distribution Visualizations</b>                                    | <b>55</b> |
| <b>B</b> | <b>Annex</b>                                                                       | <b>57</b> |

## Abstract

This project presents the design, implementation and evaluation of a portable, microcontroller-based impedance spectroscopy platform that not only measures a frequency-dependent impedance spectrum, but also interprets it directly on the device. A multisine excitation signal, optimised for a low crest factor, is generated by the DAC of an STM32H755ZI-Q microcontroller and applied to a device under test (DUT) through an analog front-end built around AD8421 instrumentation amplifiers. The resulting voltage and current signals are acquired synchronously by two timer-triggered, DMA-driven ADC channels and converted into a magnitude and phase spectrum at 15 discrete frequencies by an on-chip frequency-domain analysis stage. This spectrum is then processed by a two-stage machine-learning cascade running on the same microcontroller: a Random Forest classifier first identifies the equivalent-circuit topology (resistor, capacitor, series RC or parallel RC), and a topology-specific 1D convolutional neural network subsequently estimates the corresponding component values  $R$  and  $C$ . A host-side graphical user interface is used to visualise the measured spectra in real time (Nyquist and Bode representations), to validate measurement quality and to build the labelled training dataset.

On a held-out set containing component combinations that were never seen during training, the deployed cascade reaches a topology classification accuracy of 98.75 % and an end-to-end mean absolute error of approximately  $186\,\Omega$  for  $R$  and  $5.1\,\text{nF}$  for  $C$ , at a worst-case inference time of about 13 ms and a flash footprint of 17.5 % of the available memory. The work therefore demonstrates that autonomous, in-field equivalent-circuit parameter estimation is feasible on a low-cost embedded platform without a PC in the loop. Such a system is of interest wherever impedance-based diagnosis must be performed continuously or outside a laboratory, for example in battery state-of-health monitoring, bioimpedance measurement, corrosion and coating inspection, sensor characterisation and cable fault diagnosis.

## Member Responsibilities

*Remark: The responsible for each part needs to ensure that this part is done correctly until the deadline. He is NOT the only one working on that part.*

| Member                    | Contribution                                                                                                                                                         |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tuan Minh Le</b>       | Hardware design, Data acquisition, Model deployment.<br>Writing report (Chapter 1.10, 2.7, 3.1, 3.4)                                                                 |
| <b>Minh Quang Vu</b>      | Hardware design, Firmware design (Multi-sine, FFT), Impedance measurement analysis.<br>Writing report (chapter 1.1 to 1.7, chapter 2.1, 3.1, 3.2)                    |
| <b>Gia Khanh Nguyen</b>   | Hardware design, Firmware design (Program flow, Peripheral).<br>Writing report (Chapter 2.2, 2.3)                                                                    |
| <b>Khoi Nguyen Nguyen</b> | Data acquisition, Dataset preparation, GUI design, Machine/Deep learning model design, Model training.<br>Writing report (Chapter 1.8, 1.9, 2.4, 2.5, 2.6, 3.3, 3.5) |

# Introduction

## Motivation & Problem

Impedance spectroscopy (IS) is one of the few measurement techniques that can characterise a material, a sensor or an electrochemical cell in a completely non-destructive way. Because different physical processes inside a device under test (DUT) possess different characteristic time constants, they become visible in different regions of the frequency axis. Evaluating the complex impedance  $Z(f)$  over a range of frequencies therefore makes it possible to separate and quantify effects that a single DC or single-frequency AC measurement cannot resolve. This property explains the wide adoption of IS in battery state-of-charge and state-of-health diagnosis, bioimpedance and tissue characterisation, corrosion and coating evaluation, sensor testing and cable fault detection.

In practice, however, the interpretation of an impedance spectrum rarely stops at the raw curve. The measured spectrum is usually mapped onto an *equivalent circuit model*, i.e. a small network of passive elements whose parameters carry the actual physical meaning that the user is interested in. Extracting these parameters is classically done offline, by complex non-linear least-squares fitting on a PC. This step requires the operator to select a circuit topology beforehand, to supply reasonable initial values, and to supervise the convergence of the fit. It is therefore an expert-driven, computationally demanding and inherently non-autonomous procedure.

The instrumentation side suffers from a comparable restriction. Commercial impedance analysers and RLC meters deliver excellent accuracy, but they are bulky, cost on the order of tens of thousands of euros, and require the DUT to be brought into the laboratory. This is incompatible with the very applications that would benefit most from impedance spectroscopy, namely continuous condition monitoring, in-field diagnosis and integration into a compact product. Two further constraints appear as soon as such a measurement is to be embedded. First, the classical sequential sine sweep needs a long total acquisition time, which conflicts with the time-invariance requirement of drifting systems. Second, a resource-constrained microcontroller offers only limited ADC resolution, memory and computational throughput, so both the excitation strategy and the parameter-extraction algorithm must be chosen with those limits in mind.

The problem addressed in this project therefore combines a measurement and an inference challenge:

- **Measurement:** how can a fast, sufficiently accurate impedance spectrum be acquired using only the peripherals that are already integrated into a general-purpose microcontroller, without the analog complexity of an auto-balancing bridge or the cost of a dedicated instrument?
- **Inference:** how can the equivalent-circuit topology and its component values be recovered from that spectrum *on the device itself*, within a few tens of kilobytes of working memory and a few milliseconds of execution time, and without an expert-supplied initial guess?

The approach followed in this work answers the first question with a crest-factor-optimised multisine excitation combined with the I–V measurement method and fully digital AC analysis, and the second question with a data-driven two-stage cascade: a Random Forest classifier that determines the circuit topology, followed by topology-specific 1D convolutional neural networks that regress the corresponding parameters. Replacing iterative curve fitting by a trained model turns parameter estimation into a fixed-cost, deterministic operation, which is precisely what an embedded target requires. The complete system is realised on an STM32H755ZI-Q Nucleo-144 board with a discrete analog front-end, and is validated against reference passive networks with known nominal component values.

## Report Outline

The remainder of this report is organised as follows.

**Chapter 1 – Theoretical Background** establishes the foundations on which the design decisions of this project rest. It first introduces the principle of impedance spectroscopy and the conditions (linearity, stability, causality and finiteness) that a valid measurement must satisfy. It then reviews the available measurement platforms and measurement methods, compares analog and digital signal processing for impedance extraction, and discusses excitation signal types together with the crest-factor and frequency-bin optimisation of multisine signals. The chapter closes with the acquisition and synchronisation requirements, the AC analysis algorithms used to obtain magnitude and phase, the feature representation derived from the resulting spectrum, and the theoretical basis of the Random Forest and 1D-CNN models as well as the available options for deploying such models on a microcontroller.

**Chapter 2 – System Design and Implementation** translates those trade-offs into a concrete realisation. It presents the overall architecture and justifies the selected measurement method, excitation signal, synchronisation strategy and deployment path. It then details the analog front-end (reference generation, DAC conditioning stage, device under test and ADC front-end), the firmware structure based on timer-triggered DMA transfers, and the host-side graphical user interface used for real-time visualisation, validation and dataset recording. The second half of the chapter covers the machine-learning side: dataset construction and scaling, the prevention of data leakage, the two-stage cascade architecture, and the training and embedded deployment of the resulting models.

**Chapter 3 – Results and Evaluation** defines the evaluation methodology and reports the achieved performance on three levels: the accuracy of the impedance measurement itself, expressed as the relative complex error against theory; the accuracy of the classification and regression stages, including the cost of imperfect topology routing; and the resource utilisation and worst-case execution time of the deployed model on the target microcontroller. The chapter ends with a discussion of the limitations of both the measurement hardware and the machine-learning approach.

**Chapter 4 – Conclusion and Outlook** summarises the main contributions and results of the project and outlines the directions in which the system could be extended.

The **Annex** documents the Python data pipeline in more detail, including the dataset format, the cleaning and feature-preparation steps, the classifier and regressor implementations, the validation strategy and the exported result files. **Appendix A** finally collects the complete relative-complex-error distributions over the full frequency grid for all measured component

## 1 Theoretical Background

### 1.1 Fundamentals of Impedance Spectroscopy

Impedance Spectroscopy (IS), often referred to in the electrochemical context as Electrochemical Impedance Spectroscopy (EIS), is a measurement technique that characterizes the electrical behavior of a material, sensor, or device under test (DUT) by evaluating its impedance over a range of excitation frequencies [1].

The measurement procedure consists of applying a small-amplitude, time-varying excitation signal, which typically is a voltage  $v(t)$  (or, equivalently, a current  $i(t)$ ), to the Device Under Test (DUT) and measuring the resulting response signal, namely the current  $i(t)$  (or voltage  $v(t)$ ). Because both signals are analyzed in the frequency domain, the impedance at a given frequency  $f$  is obtained as the ratio of their Fourier transforms:

$$Z(f) = \frac{V(f)}{I(f)} \quad (1)$$

where  $V(f) = |V|\angle\varphi_v$  and  $I(f) = |I|\angle\varphi_i$  are the complex phasors of the voltage and current signals at frequency  $f$ , respectively. The impedance is therefore a complex-valued quantity,  $Z(f) = |Z|\angle\varphi_z$ , which encodes both a magnitude, reflecting the DUT's opposition to current flow; and a phase, representing the time or energy delay between excitation and response caused by capacitive or inductive components.

By repeating this evaluation across a set of frequencies rather than at a single operating point, an impedance spectrum  $Z(f)$  is obtained. This is the defining feature of the technique and the reason for the name "spectroscopy".

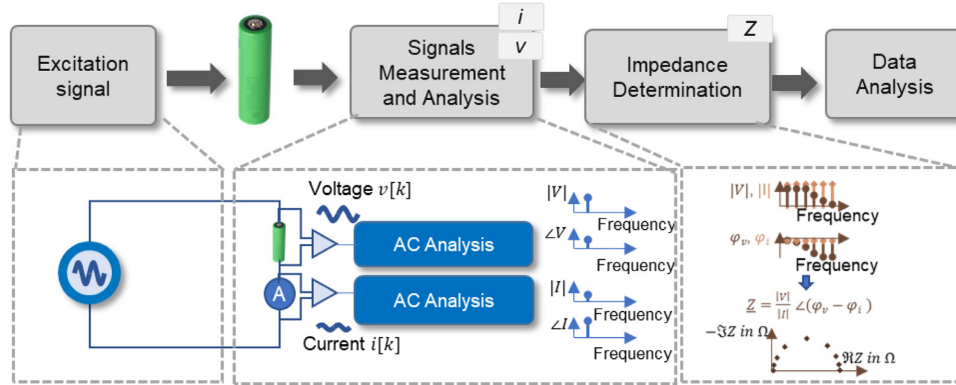


Figure 1: Principle of impedance spectroscopy measurement. [1]

### 1.1.1 Material Characterization Capabilities of IS

Different physical and electrochemical phenomena occurring within a DUT have characteristic time constants, and therefore reveal themselves at different, distinguishable frequency ranges. For instance, charge-transfer reactions at an electrode interface, and double layer capacitance effects dominate the impedance response in a different frequency components [1]. Because a frequency sweep probes the DUT at many time scales simultaneously (or sequentially), EIS can separate and quantify these underlying processes, which something a single DC or single-frequency AC measurement cannot achieve. This is the fundamental reason EIS is used not merely to determine an electrical parameter (e.g., a single resistance value), but to infer physical or chemical properties of the material or system, such as electrode kinetics, corrosion state, cell composition, or tissue characteristics.

Owing to this diagnostic capability, EIS is applied across a wide range of fields, including:

- **Biomedicine and biology** – bioimpedance spectroscopy for tissue characterization and body composition analysis.
- **Batteries and fuel cells** – state-of-charge (SoC) and state-of-health (SoH) diagnosis, aging characterization.
- **Corrosion and coatings** – material and surface degradation testing.
- **Sensor characterization and material testing** – dielectric and electrochemical sensor evaluation.
- **Cable and wire diagnosis** – fault detection in the frequency domain.

Each of these applications imposes a different required excitation frequency range and impedance measurement range, as illustrated in Fig. 2.

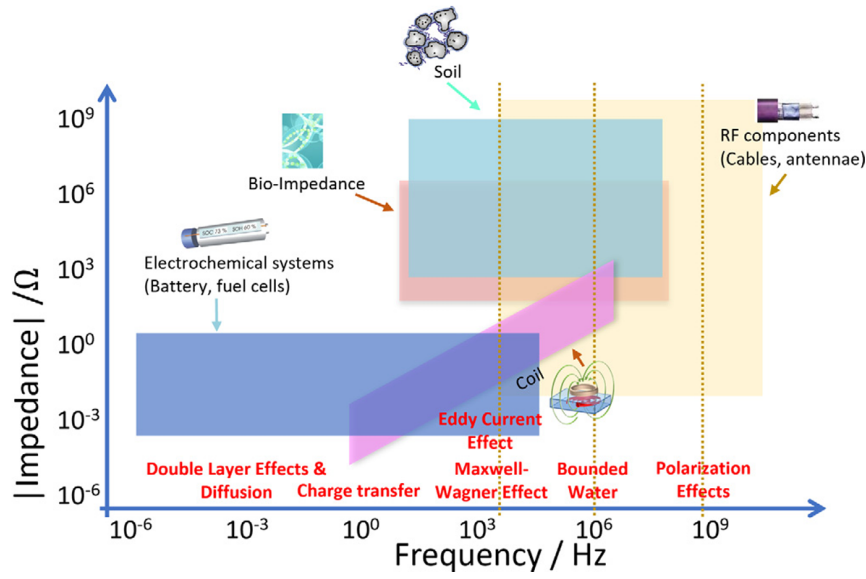


Figure 2: Frequency ranges, impedance ranges and corresponding effects for selected applications of impedance spectroscopy. [1]

## 1.2 Requirements for Consistent EIS Measurements

For an impedance spectrum to be physically meaningful and reproducible, the measurement process must satisfy four fundamental conditions: linearity, stability (time-invariance), causality, and finiteness [1]. If any of these conditions is violated, systematic deviations are introduced into the measured spectrum, and subsequent interpretation (e.g., equivalent-circuit fitting or data-driven analysis) becomes unreliable. This section explains each requirement and how measurement consistency can be verified.

### 1.2.1 Linearity

A system is linear if its response to an input signal is strictly proportional to that input, and if the response to a sum of inputs is the sum of the responses to each individual input (superposition). For a multisine excitation signal composed of  $K$  sinusoidal tones,

$$u(t) = \sum_{k=1}^K M_k \cos(\omega_k t + \varphi_k) = \sum_{k=1}^K A_k \cos(\omega_k t) + B_k \sin(\omega_k t), \quad (2)$$

with  $A_k = M_k \cos(\varphi_k)$  and  $B_k = -M_k \sin(\varphi_k)$ , a linear system  $h(t)$  produces a response that remains a sum of tones at the *same* frequencies, only with modified (frequency dependent) amplitude and phase:

$$y(t) = \sum_{k=1}^K A'_k \cos(\omega_k t) + B'_k \sin(\omega_k t). \quad (3)$$

Most electrochemical and biological systems are inherently non-linear. If the excitation amplitude is too large, the DUT is driven outside its linear operating region, and the response signal develops harmonic distortion: new frequency components appear at integer multiples of the excitation frequencies that were not present in the excitation signal itself. Consequently, the excitation amplitude must be chosen large enough to guarantee an adequate signal-to-noise ratio, yet small enough to avoid violating the linearity condition [1].

### 1.2.2 Stability (Time-Invariance)

A measurement is only valid if the DUT's internal state does not change during the acquisition of the spectrum. For example, the system must be time-invariant (stationary) throughout the excitation period. If the DUT's state drifts (such as a battery's state of charge (SoC) changing due to net charge injected by the excitation signal, or thermal drift affecting a sensor), different points of the impedance spectrum end up corresponding to different underlying states of the DUT. The resulting spectrum is then a distorted composite of multiple snapshots rather than a single consistent measurement.

To minimize this risk, the total measurement duration should be kept as short as possible, which is a key motivation for adopting multi-frequency excitation signals (e.g., multisine) instead of a sequential single-frequency sine sweep; the former can reduce total acquisition time by an order of magnitude for a comparable frequency coverage [1].

Stability can also be interpreted energetically: the output response must not grow unbounded with time, i.e., the impedance magnitude must remain bounded over the full (real, positive) frequency range:

$$|Z(\omega)|^2 < A, \quad 0 < \omega < \infty \quad (4)$$

for some finite constant  $A$ .

### 1.2.3 Causality

Causality requires that the measured response depends only on the present and past values of the excitation signal, never on future values. In the context of impedance spectroscopy, this condition is stricter than the generic Linear-Time-Invariant (LTI) definition: it also requires that any pre-existing relaxation phenomena (e.g., residual charge/mass transport effects from a previous measurement or operating condition) have fully decayed before the excitation is applied, so that the measured response can be unambiguously attributed to the applied perturbation alone. Systems exhibiting hysteresis are particularly problematic in this respect, since their response also depends on excitation history. For this reason, a sufficient rest (relaxation) period, on the order of 15 to 30 minutes for electrochemical systems such as batteries, is typically required before starting a measurement [1].

## 1.3 Classification of Measuring Platforms

Several categories of measurement platforms are available for impedance spectroscopy, ranging from bulky, highly accurate laboratory instruments to compact, portable embedded systems, as described in Table 1. The choice of platform has a direct impact on development effort, cost, portability, and achievable measurement accuracy [1].

Table 1: Qualitative comparison of measurement platforms for impedance spectroscopy ( adapted from [1]).

| Platform              | Category      | Size    | Dev. effort             | Accuracy  | Rel. cost  |
|-----------------------|---------------|---------|-------------------------|-----------|------------|
| Impedance analyzer    | Lab-exclusive | Bulky   | Ready-to-use            | Highest   | Very high  |
| RLC meter             | Lab-exclusive | Bulky   | Ready-to-use            | Very high | High       |
| c-DAQ-based           | Portable      | Compact | PC-based programming    | High      | Medium     |
| FPGA-based            | Portable      | Small   | HDL + PCB + embedded SW | High      | Medium-low |
| Microcontroller-based | Portable      | Small   | PCB + embedded SW       | Moderate  | <b>Low</b> |

- **Impedance analyzers and RLC meters** are dedicated, ready-to-use laboratory instruments. They provide the highest measurement accuracy but are bulky, expensive (on the

order of tens of thousands of euros), and require the DUT to be brought to the laboratory, which precludes in-field or continuous monitoring applications.

- **Vector Network Analyzers (VNA)** are similarly lab-exclusive, high-accuracy instruments, primarily suited for high-frequency (RF) impedance matching applications.
- **Compact Data Acquisition systems (c-DAQ)** combine off-the-shelf analog input/output modules with PC-based software (e.g., LabVIEW, Python) to realize a portable, moderately accurate measurement system without requiring custom hardware design. Their main limitations are a restricted frequency range (typically below 100 kHz) and a cost that, while lower than laboratory instruments, is still significant.
- **FPGA-based systems** achieve high measurement accuracy through fully customizable, parallel hardware architectures. However, they require hardware description language (HDL) programming (e.g., Verilog) in addition to embedded software development, and typically also require external dedicated chips (DDS for signal generation, I/Q demodulators for signal analysis) to compensate for the complexity of implementing signal generation/acquisition purely in logic. This results in higher power consumption, higher cost, and a substantially more complex development process compared to microcontroller-based alternatives [1].
- **Microcontroller-based (MCU-based) systems** offer a lower-cost, lower-power, and more compact alternative to FPGAs. Because signal generation (DAC) and acquisition (ADC) peripherals are typically already integrated on-chip, MCU-based systems can often be realized without additional dedicated hardware, and are programmed using standard embedded software (e.g., C) rather than HDL. The trade-off is a somewhat reduced measurement accuracy compared to FPGA or c-DAQ solutions, primarily due to limited ADC resolution/speed and processing throughput.

## 1.4 Embedded Measurement

Implementing an impedance measurement system on an embedded platform requires evaluating which measurement method offers the best trade-off between hardware complexity and signal processing requirements.

### 1.4.1 Comparison of Measurement Methods

Regardless of the underlying hardware platform, impedance measurement systems are generally built around one of the following core principles [1]:

- **I–V method:** An excitation voltage (or current) is applied to the DUT, and the resulting current (or voltage) is measured simultaneously. Frequency-domain analysis of both signals subsequently yields the complex impedance. This method is highly effective for frequencies below approximately 10 MHz.
- **RF I–V method:** Conceptually similar to the standard I–V method, but optimized for radio-frequency (RF) ranges and for DUTs with impedances close to a characteristic reference (e.g., 50  $\Omega$ ).
- **Bridge method:** Balances the unknown DUT impedance against a reference impedance until the current through the detector branch is nullified. The traditional reliance on manual tuning makes this method difficult to automate within autonomous embedded systems.

- **Resonance method:** Excites an LC tank circuit containing the DUT with a variable frequency and infers the impedance properties from the resonant peak of the response.
- **Auto-Balancing Bridge (ABB) method:** Replaces the manual adjustment of the bridge with an active, automatically phase-shifted feedback loop, achieving exceptional accuracy at the expense of a highly complex analog front-end.
- **Vector Network Analysis (VNA):** Injects an incident electromagnetic wave and analyzes the reflected and transmitted waves, which is essential for high-frequency in GHz-range characterization requiring strict impedance matching.

For general-purpose impedance spectroscopy applications operating in the Hz to MHz range, such as electrochemical systems, material characterization, or bioimpedance, both the auto-balancing bridge and the standard I–V method offer suitable accuracy. However, the I–V method presents a distinct advantage for resource-constrained embedded systems. Because it relies on simultaneous voltage and current acquisition, it can interface directly with standard digital-to-analog (DAC) and analog-to-digital (ADC) converters without requiring complex analog balancing networks [1]. Consequently, the I–V method serves as a highly suitable foundation for portable, microcontroller-based impedance analyzers.

#### 1.4.2 Digital vs. Analog Signal Processing for Impedance Extraction

Within the I–V method, the extraction of amplitude and phase from the acquired voltage and current samples can be performed either using analog circuitry or digitally through microcontroller-based signal processing.

In the analog approach, demodulation hardware such as lock-in amplifiers, I/Q demodulators, or gain-phase detectors performs the frequency extraction directly at the hardware level, yielding a slowly varying DC voltage that directly represents the real/imaginary (or magnitude/phase) components at the frequency of interest. As a result, the subsequent ADC sampling stage no longer needs to resolve the instantaneous AC waveform, and no dedicated AC analysis algorithm (Section 1.7) is required; this also relaxes the strict DAC–ADC timing-synchronization requirements discussed in Section 1.6, since only a stable DC value needs to be captured.

This simplicity, however, comes at the cost of scalability: because each demodulation circuit is inherently tuned to a single frequency, multi-frequency excitation (e.g., multisine, as adopted in Section 1.5) would require replicating the analog demodulation hardware for every excited frequency, rapidly increasing circuit complexity, board size, and power consumption.

Digital signal processing avoids this limitation entirely. Here, the ADC samples the raw voltage and current waveforms directly, and the extraction of amplitude and phase for all excited frequencies is performed numerically by the microcontroller (e.g., via FFT). This shifts the demodulation tasks to reusable software, at the cost of requiring precise DAC–ADC synchronization to avoid spectral leakage.

Consequently, combining high accuracy, low power, and a compact footprint with its inherent compatibility with multi-frequency excitation, digital AC analysis represents a highly scalable and general-purpose solution for resource-constrained embedded impedance spectroscopy systems operating up to approximately 10 MHz [1].

#### 1.4.3 Overall Block Diagram of the I–V-Based Architecture

The hardware architecture of a general I–V-based embedded impedance spectroscopy measurement is structured into different functional stages, as illustrated in Figure 3. The signal chain flows through the following functional blocks:

1. **Excitation Voltage Signal Generation:** The AC excitation waveform is digitally synthesized by the microcontroller using internal peripherals, specifically a Digital-to-Analog Converter (DAC) or Pulse Width Modulation (PWM).
2. **Offset Removal:** Since single-supply microcontroller peripherals output unipolar signals, this stage eliminates the DC offset using a high-pass filter or an analog DC subtractor, producing a symmetrical AC voltage waveform.
3. **Voltage-Controlled Current Source (Optional VCCS):** Indicated by the dashed outline in Figure 3, this block is an optional hardware stage that dictates the excitation mode:
  - **Galvanostatic mode:** The VCCS is engaged to convert the conditioned AC voltage into a precise, load-independent AC current before driving the Device Under Test (DUT).
  - **Potentiostatic mode:** The VCCS is bypassed, allowing the conditioned AC voltage signal to drive the DUT directly.
4. **Current-to-Voltage (I–V) Conversion:** To perform the I–V measurement, the current passing through the DUT must be translated back into a voltage signal. This is achieved using either a passive shunt resistor or an active transimpedance (I–V) converter.
5. **Signal Conditioning (Offset & Amplification):** Both the DUT AC voltage drop and the converted AC current voltage signal are routed to this stage. Utilizing differential operational amplifiers or instrumentation amplifiers, the signals are level-shifted and amplified to maximize the dynamic range of the ADCs.
6. **Microcontroller Acquisition and Digital AC Analysis:** The AC voltage and AC current signals are acquired simultaneously via two independent ADC channels. The digitized samples are then processed by the microcontroller's internal AC analysis core to calculate the magnitude and phase, ultimately yielding the complex impedance of the DUT.

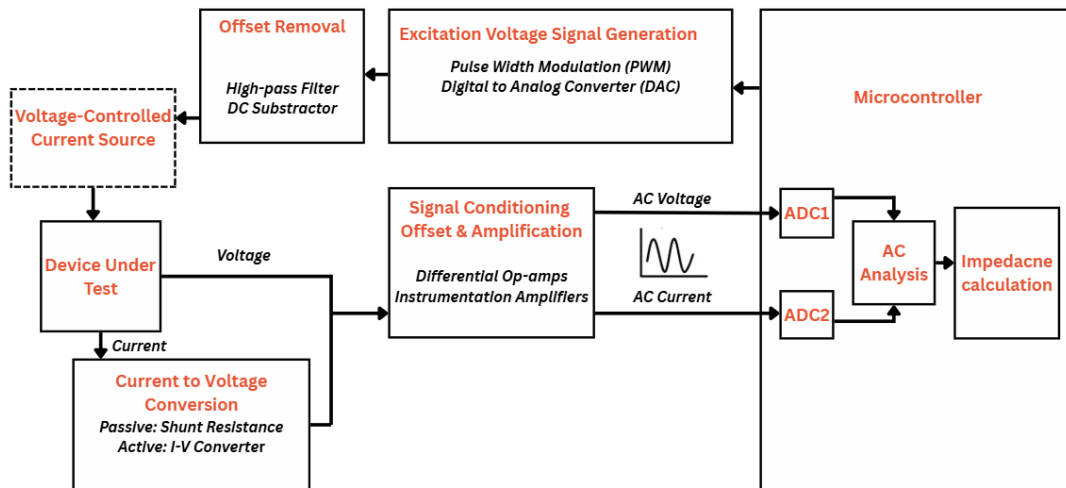


Figure 3: Block diagram of the I–V based embedded measurement system architecture.

## 1.5 Excitation Signals

The choice and design of the excitation signal is one of the most critical steps in the development of an embedded impedance spectrometer: it directly determines measurement duration,

achievable signal-to-noise ratio, and also, as discussed in Section 1.2, whether the linearity and stability requirements can be satisfied [1].

### 1.5.1 Available Excitation Signal Types

Excitation signals for impedance spectroscopy can be broadly classified based on their characteristics. While numerous signaling techniques exist, embedded applications typically focus on the following primary approaches:

- **Sine sweep:** A single sinusoidal tone is applied and analyzed at each frequency of interest sequentially. It offers the highest amplitude and power spectral density per excited frequency, resulting in excellent energy efficiency and signal-to-noise ratio (SNR). However, it requires a long total measurement time, which directly conflicts with the stability requirement for time-varying under test (DUTs).
- **Binary sweep:** A binarized, square-wave alternative to the conventional sine sweep. This approach is highly useful when the embedded hardware lacks a DAC and must rely entirely on binary digital outputs (e.g., GPIO or basic PWM), though it suffers from reduced spectral amplitude at the fundamental frequency compared to pure sinusoids.
- **Multisine:** A simultaneous superposition of  $N_f$  sinusoidal tones at arbitrarily selectable frequencies, generated concurrently rather than sequentially. Because the frequencies and their respective count can be freely customized, the measurement duration is significantly reduced while maintaining high energy and favorable spectral amplitude levels. This method is well-known for rapid impedance spectroscopy.

Beyond these primary methods, other specialized excitation techniques, such as wideband impulses, Pseudo-Random Binary Sequences, Chirps, and Discrete Interval Binary Sequences, which are extensively documented in the literature [1]. A qualitative comparison of these signal types, in terms of frequency distribution, typical measurement duration, amplitude level, and energy efficiency, is summarized in Table 2.

Table 2: Qualitative comparison of excitation signal types (adapted from [1]).

| Signal       | Freq. distrib. | Duration | Amplitude level | Energy eff. |
|--------------|----------------|----------|-----------------|-------------|
| Sine sweep   | Arbitrary      | Long     | High            | High        |
| Binary sweep | Arbitrary      | Long     | High            | High-medium |
| Impulse      | Dense          | Short    | Low             | Low         |
| PRBS/MLBS    | Dense          | Short    | Low             | Low         |
| Chirp        | Dense          | Short    | Medium          | High        |
| Signum chirp | Dense          | Short    | Medium          | Medium      |
| Multisine    | Arbitrary      | Short    | Medium-high     | High        |
| DIBS         | Arbitrary      | Short    | Medium-high     | Medium      |

In summary, multisine excitation serves as an ideal, general-purpose solution for resource-constrained embedded impedance spectroscopy systems. By utilizing DACs commonly integrated into modern microcontrollers, it provides an optimal compromise between rapid measurement execution, which is essential for preserving the time-invariance of drifting systems; and high energy efficiency.

### 1.5.2 Optimization of Multisine Signals

While multisine signals offer a significant reduction in measurement time, their practical implementation in embedded impedance spectroscopy requires an optimization process.

A major challenge in generating multisine excitation signals is their susceptibility to a high Crest Factor (CF), is defined as the ratio of the absolute peak value to the root-mean-square (RMS) value of the time-domain signal  $s[n]$ :

$$CF = \frac{\|s[n]\|_\infty}{\|s[n]\|_2} = \frac{\max_n |s[n]|}{\sqrt{\frac{1}{N} \sum_{n=1}^N s^2[n]}}. \quad (5)$$

Because a multisine signal is a linear superposition of multiple phase-shifted sinusoids, constructive interference can cause the instantaneous time-domain signal to exhibit massive voltage or current peaks. In resource-constrained embedded systems, this peak-power issue introduces severe limitations, such as signal clipping, since these unoptimized peaks will inevitably exceed the fixed voltage range of the integrated DAC or ADC (e.g., 0–3.3 V or 0–5 V).

Although this peak-power problem can theoretically be resolved through normalization by dividing the entire time-domain waveform by its absolute peak value. However, this approach leads to severe dynamic range degradation (loss of ENOB) and a degraded SNR. To avoid this excessive reduction in signal power, it is highly recommended to optimize the crest factor first as stated in [1] and shown in Figure 4.

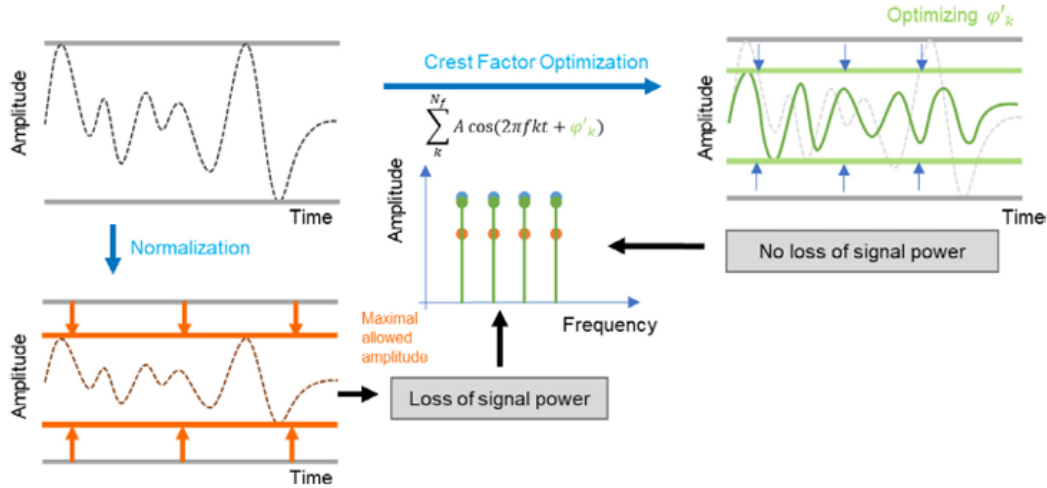


Figure 4: Comparison between non-optimized versus optimized multi-sine signal. [1]

Mathematically, given  $N_f$  frequencies,  $N$  samples at a sampling rate of  $F_s$ , and elementary sinewaves defined by their amplitudes  $A_k$ , frequencies  $f_k$ , and phase values  $\phi_k$ , the crest factor optimization minimizes the ratio between the peak  $\ell^\infty$ -norm and the RMS  $\ell^2$ -norm of the resulting time-domain signal  $s[n]$ , as formulated in Eq. (7):

$$s[n] = \sum_{k=1}^{N_f} A_k \cos\left(2\pi \frac{f_k}{F_s} n + \phi_k\right) \quad (6)$$

$$\arg \min_{(\phi_k)_{k=1, \dots, N_f}} \frac{\left\| \sum_{k=1}^{N_f} A_k \cos\left(2\pi \frac{f_k}{F_s} n + \phi_k\right) \right\|_\infty}{\left\| \sum_{k=1}^{N_f} A_k \cos\left(2\pi \frac{f_k}{F_s} n + \phi_k\right) \right\|_2}. \quad (7)$$

A multisine signal is generated over a finite number of samples  $N$  at a sampling frequency  $F_s$ . To guarantee that each individual sinusoid completes an integer number of periods within the measurement window, which avoids spectral leakage during the subsequent AC analysis (Section 1.7), each desired frequency  $f$  must be rounded to the nearest available frequency bin of the discrete spectrum:

$$f_{opt} = \left\lfloor f \frac{N}{F_s} \right\rfloor \frac{F_s}{N}, \quad (8)$$

where  $\lfloor \cdot \rfloor$  denotes rounding to the nearest integer. This step, referred to as frequency bin optimization, must be performed before crest-factor optimization.

## 1.6 Acquisition of Current and Voltage Signals and Synchronization

Accurate impedance determination via the I-V method (Section 1.4) requires that both the excitation signal actually realized by the DAC and the resulting current/voltage response be sampled consistently with respect to a common time base. In practice, this means the timers controlling signal generation (DAC) and signal acquisition (ADC) must be synchronized [1].

If the DAC and ADC operate from independent, unsynchronized clocks (or if the acquisition window does not align with an integer number of periods of the excitation signal), the sampled data window effectively truncates the periodic excitation signal at an arbitrary, non-periodic boundary. In the frequency domain, this manifests as spectral leakage: energy that should be concentrated at the excited frequency bins spreads into neighboring bins, corrupting the amplitude and phase estimates obtained from the subsequent AC analysis process, as illustrated in Figure 5.

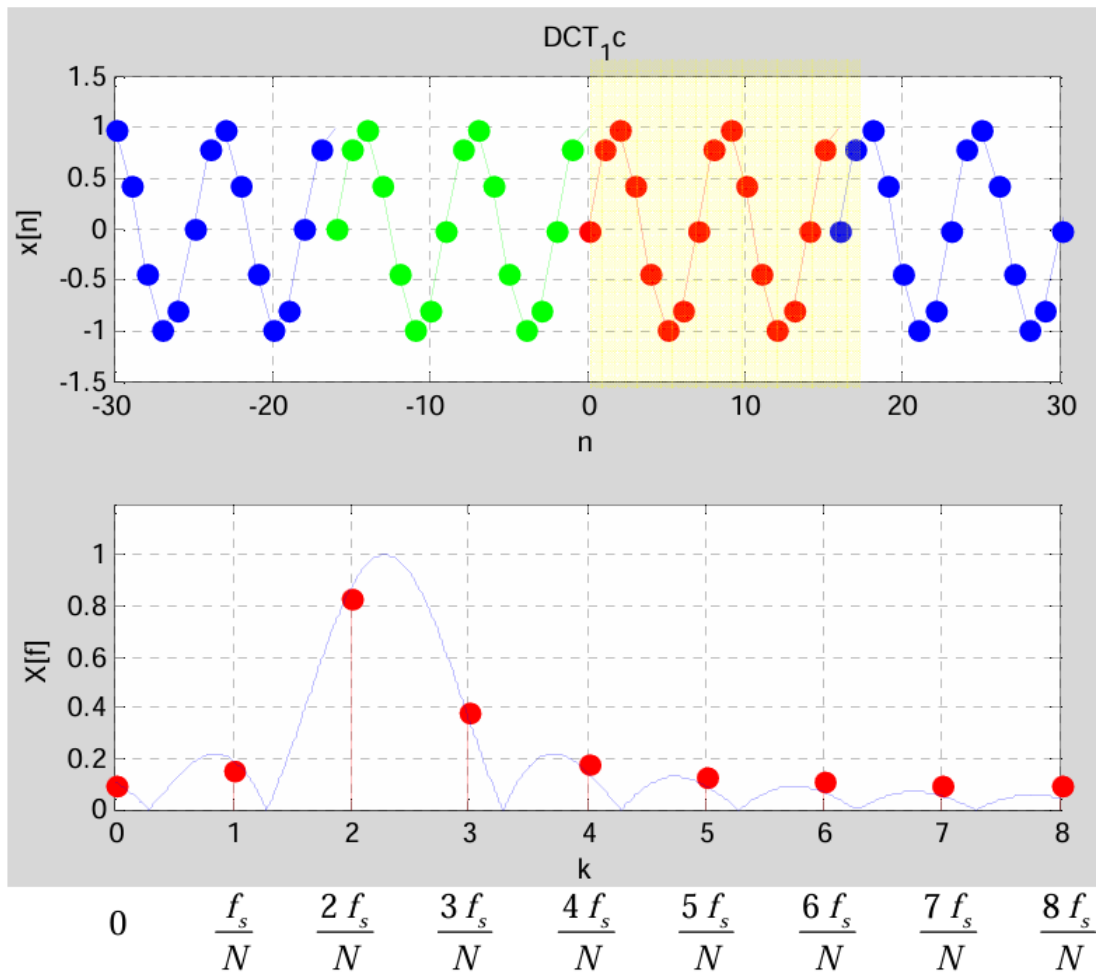


Figure 5: Leakage due to the acquisition window not aligning with an integer number of periods

On a modern microcontroller (e.g., STM32-class devices), timer-based synchronization can be realized efficiently by configuring one hardware timer to trigger the DAC conversion/output

and a second (clock-synchronized) timer to trigger ADC sampling, while using Direct Memory Access (DMA) channels to transfer generated/acquired samples to and from memory without CPU intervention. This DMA-timer architecture both guarantees precise, jitter-free timing and frees the CPU core to perform other tasks (e.g., the AC analysis of the previous measurement) concurrently with signal generation/acquisition.

## 1.7 AC Signal Analysis

Once the voltage and current signals have been synchronously acquired (Section 1.6), the final processing step before complex impedance can be computed is AC signal analysis. This stage focuses on extracting the amplitude and phase of both the voltage and current signals at each excited frequency. Because this extraction must run in real-time on a resource-constrained microcontroller, the chosen digital signal processing algorithm must carefully balance accuracy against computational complexity and memory footprint [1].

### 1.7.1 Primary Processing Methods

While several advanced signal processing techniques are documented in the literature—such as the single-frequency Goertzel filter or iterative time-domain Ordinary / Non-linear Least Squares (OLS / NLLS) sine fitting [1], embedded impedance spectroscopy systems typically evaluate performance based on the following two primary frequency-domain transforms:

- **Fast Fourier Transform (FFT):** An algorithmically accelerated implementation of the full  $N$ -point Discrete Fourier Transform (DFT) that computes all  $N/2$  frequency bins. By exploiting a radix-2 divide-and-conquer strategy, it eliminates redundant trigonometric computations. This method is highly efficient when the entire spectrum is required or when utilizing highly optimized vendor-provided hardware DSP libraries.
- **K-point DFT:** A straightforward DFT evaluated strictly at the  $K$  excited frequencies of interest. Unlike the full FFT, it discards unexcited frequency bins, making it computationally attractive when the number of excited channels  $K$  is extremely small relative to the window length  $N$  ( $K \ll N$ ).

A comparison of the asymptotic computational complexity and memory footprint for these two primary methods is summarized in Table 3.

Table 3: Computational complexity and memory usage of primary AC analysis methods (adapted from [1]).

| Method        | Computational complexity  | Memory usage           |
|---------------|---------------------------|------------------------|
| FFT (Radix-2) | $\mathcal{O}(N \log_2 N)$ | $2N$ (in-place) / $4N$ |
| K-point DFT   | $\mathcal{O}(2KN)$        | $4K + 2NK$             |

### 1.7.2 Hardware Acceleration

Many modern microcontrollers provide dedicated on-chip hardware execution units and optimized instruction sets designed to accelerate computationally intensive digital signal processing algorithms without incurring excessive power consumption or requiring expensive external hardware. Specifically, ARM Cortex-M4 and Cortex-M7 cores, such as those found in the STM32 family of microcontrollers, integrate advanced single-cycle Multiply-Accumulate (MAC) instructions alongside Single Instruction Multiple Data (SIMD) instructions to perform parallel, high-speed vector calculations in a single clock cycle[2].

To fully exploit these physical hardware features, the Cortex Microcontroller Software Interface Standard (CMSIS-DSP) library acts as a highly optimized, vendor-independent abstraction layer. By utilizing assembly-level intrinsics that directly invoke these hardware acceleration units, the CMSIS-DSP library executes real-time digital transforms, such as the Radix-2 FFT, with exceptional processing speeds and minimal CPU cycle consumption, offering a highly efficient signal processing engine for resource-constrained embedded platforms.

### 1.7.3 From AC Analysis to Impedance

Given the amplitude and phase of the voltage and current at a given excited frequency bin  $k$ , which obtained directly as complex values  $V[k]$  and  $I[k]$  from the FFT output:

$$|V[k]| = \sqrt{V_{\text{re}}[k]^2 + V_{\text{im}}[k]^2}, \quad \varphi_v[k] = \tan^{-1} \left( \frac{V_{\text{im}}[k]}{V_{\text{re}}[k]} \right), \quad (9)$$

the complex impedance  $Z[k]$  at that specific frequency is calculated using direct complex division:

$$Z[k] = \frac{V[k]}{I[k]}. \quad (10)$$

Alternatively, to avoid computationally expensive complex division on MCUs lacking a hardware floating-point unit (FPU) dedicated to complex arithmetic, the impedance is resolved through magnitude ratio and phase subtraction:

$$|Z[k]| = \frac{|V[k]|}{|I[k]|}, \quad \varphi_z[k] = \varphi_v[k] - \varphi_i[k]. \quad (11)$$

Repeating this calculation for every excited frequency bin  $k$  produces the final impedance spectrum  $Z[k]$ , which serves as the fundamental input for subsequent state monitoring and parameter estimation.

## 1.8 Feature Extraction

Before the raw EIS measurements can be fed into the classification and regression models, they must be transformed into a representation that preserves the physically meaningful information embedded in the spectrum.

### 1.8.1 From AC Signal to EIS Features

In the EIS measurement system of this project, the AC signal are swept across multiple discrete frequency points. At each frequency, the system requires the amplitude and phase quantities, then represents the complex impedance as the formula:

$$Z(\omega) = Z_{\text{real}}(\omega) + j Z_{\text{imag}}(\omega) \quad (12)$$

with two equivalent feature representations:

**Polar form.**

$$|Z| = \sqrt{Z_{\text{real}}^2 + Z_{\text{imag}}^2}, \quad \phi = \text{atan2}(Z_{\text{imag}}, Z_{\text{real}}) \quad (13)$$

**Cartesian form.**

$$Z_{\text{real}} = |Z| \cos \phi, \quad Z_{\text{imag}} = |Z| \sin \phi \quad (14)$$

Thus, a measurement sample is no longer a single meaningless number, but a sequence of points across frequencies:

- Magnitude sequence over frequency
- Phase sequence over frequency
- Or equivalently, a  $(Z_{\text{real}}, Z_{\text{imag}})$  sequence over frequency

In the current project, the data is organized into multiple **Magnitude\_i** and **Phase\_i** columns where i represent for hte frequency slots, and then concatenated into a 2-channel tensor for the deep learning model:

- **Channel 1:** Magnitude
- **Channel 2:** Phase

This accurately reflects the physical nature of EIS that the information lies in the *spectral shape across frequencies*, not in just a single isolated point.

**1.8.2 Why Preserve the Sequence Instead of Extracting Mean/Variance Statistics?**

If we only extract aggregate statistics such as mean, variance, min, and max, we heavily compress the data and that lead to lose the frequency-dependent structure. For EIS, this is a critical loss because:

- **Knee frequency is a local phenomenon along the frequency axis:** The knee frequency appears as a transition region in the circuit response . This is a *geometric break point* that can only find in the Nyquist/Bode plot, which cannot be fully described by mean or variance.
- **Two spectra with different shapes can share the same mean/variance:** Two samples might have the same average magnitude, but one decreases steadily with frequency while the other exhibits a distinct bending region. Physically, these two samples represent different components, but low-order statistics fail to distinguish them effectively.

Therefore, in the pipeline of this project, the frequency sequence is preserved in the measurement order so that the model can exploit the sequential structure rather than relying mainly on global statistical features.

**1.9 Machine Learning & Deep Learning Fundamentals**

This section reviews the theoretical foundations of two model families commonly used for spectral classification and regression tasks: Random Forest and 1D-CNN. The goal here is purely conceptual to establish the mathematical and intuitive basis of each approach before any model selection is discussed.

**1.9.1 Classification Theory with Random Forest**

Random Forest is an ensemble of multiple decision trees trained on bootstrap samples and random feature subsets [3]. Each tree partitions the feature space using threshold rules such as:

- **Magnitude\_k** < threshold
- **Phase\_m** ≥ threshold

The final prediction is the majority vote of the entire forest.

In general, decision-tree ensembles tend to be well-suited to problems with the following characteristics:

- **Discrete, regionally separable data:** When classes generate fairly distinct response bands across the feature space, decision trees are effective, since class boundaries can often be described by a sequence of threshold conditions.
- **Robustness to noise:** Each individual tree may be affected differently by noise, but averaging the votes across many trees tends to significantly mitigate random noise.
- **Insensitivity to scale normalization:** Decision trees rely on the order and threshold of feature values rather than on Euclidean distances, unlike SVM-RBF or pure MLP networks. As a result, Random Forest models are generally more robust when input features (e.g. magnitude and phase) have very different value scales.

These properties make Random Forest a common and strong baseline for classification tasks involving structured, threshold-separable feature spaces.

### 1.9.2 Regression Theory with 1D-CNN

A 1D-CNN differs from a 2D-CNN in the nature of its input:

- **2D-CNN:** processes images with two spatial dimensions (height, width).
- **1D-CNN:** processes 1D sequences along a time/frequency axis.

For sequential data where each point along an axis (e.g. frequency) carries multiple feature channels, the 1D convolution performs:

$$y_k(i) = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{M-1} w_{k,c,m} x_c(i+m) + b_k \quad (15)$$

where:

- $y_k(i)$ : the output value of the  $k$ -th filter at position  $i$
- $k$ : the index of the output channel (filter)
- $i$ : the sliding position along the sequential axis
- $C_{\text{in}}$ : the total number of input channels
- $M$ : the kernel size (window length)
- $w_{k,c,m}$ : the trainable weight of the kernel
- $x_c(i+m)$ : the input value at channel  $c$ , position  $i+m$
- $b_k$ : the learned bias term for the  $k$ -th filter

#### Physical meaning:

- The kernel learns local variation patterns along the sequential axis.
- It can be viewed as learning filters analogous to 1st and 2nd order derivatives with respect to the axis variable (slope, curvature).

Consequently, 1D-CNNs are generally effective at detecting transition regions and non-linear inflection points within a sequence [4][5]. For continuous-valued regression targets, this is often considered advantageous over tree-based regressors, since the CNN's output is a smooth function with respect to its learned parameters.

### 1.9.3 Limitations of Traditional Models

**Staircase Effect of Random Forest Regressor:** Each leaf of a tree returns a fixed average value. When ensembling multiple trees, the output still retains a step-wise (staircase) nature:

$$\hat{y}(x) \approx \sum_l c_l \mathbf{1}_{\{x \in R_l\}} \quad (16)$$

As a result, tree ensembles can struggle to represent smooth, continuous relationships across a full input range. Typical consequences include:

- Good accuracy in well-sampled (seen) regions.
- Higher error in continuous transition regions or for unseen/extrapolated values.

**MLPs Easily Struggle When Data Has Strong Local Structures:** Fully connected MLPs treat all features as flat and do not directly exploit positional or spectral proximity. To learn local patterns the way a CNN does, an MLP typically requires:

- More neurons
- More data
- Stronger regularization

Otherwise, it tends to either underfit or overfit.

**Data Leakage Problems:** In general, if train/test splits contain highly similar or repeated samples (e.g. the same underlying parameter values measured multiple times), a model can achieve artificially high scores without reflecting true extrapolation capability. This makes it important to consider how data is split - e.g. by value groups rather than by row - in order to accurately evaluate a model's generalization capability.

## 1.10 Machine Learning Deployment

Several options exist for deploying a trained model on an STM32 target. Table 4 summarizes the pros and cons of some possible options in this project's context.

| Option                   | Pros                                                                                                                                                                        | Cons                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| X-CUBE-AI (ST official)  | Official ST tool, built-in quantization, optimized kernels for STM32                                                                                                        | Designed around a single network graph; does not natively support a Random Forest + multiple independent per-class CNN cascade architecture          |
| TensorFlow Lite Micro    | Widely used, supports quantization, portable across MCU vendors                                                                                                             | No native Random Forest support; runtime footprint and integration overhead not justified for this small cascade model                               |
| CMSIS-NN                 | Optimized, hardware-accelerated NN kernels for Cortex-M                                                                                                                     | Covers only the CNN layers, not the Random Forest stage; would require a second, separate pipeline                                                   |
| Custom C code generation | Single dependency-free header file; supports the exact cascade architecture (RF + per-class CNN) as-is; no external runtime library needed; full control over memory layout | No automatic quantization and auto optimization for the target hardware; requires much time and effort to optimize the runtime and size of the model |

Table 4: Comparison of embedded ML deployment options considered for this project.

From Table 4, we can see that there is a support tool X-CUBE-AI for ML deployment with helpful built-in features, but it is only suitable for a single network model. Similarly, there are TensorFlow Lite Micro and CMSIS-NN, two frameworks which have specialized functions that help optimize the deployment, but they can still support neural network model or Random Forest model, not both. Meanwhile, custom C code generation approach offers a better flexibility to deploy all kinds of models on the target controller, but it will require much more effort to optimize the deployment without any support framework.

## 2 System Design and Implementation

This chapter presents the realization of the measurement and inference system, translating the theoretical trade-offs discussed in Chapter 1 into a specific hardware and software design. Section 2.1 gives an overview of the overall proposed system architecture. Section 2.2 then detail the analog front-end and the microcontroller’s signal generation/acquisition chain, while Section 2.3 describes the resulting firmware execution flow. The remainder of the chapter turns to the machine-learning side of the system: Section 2.5 describes how raw measurements are transformed into a training dataset, Section 2.6 presents the two-stage classification and regression architecture, and Section 2.7 covers model training and its deployment onto the target microcontroller.

### 2.1 System Architecture Overview

The measurement and inference system developed in this project consists of three main subsystems: (i) an embedded measurement front-end, responsible for generating the excitation signal and acquiring the DUT’s voltage and current response; (ii) an on-device machine-learning inference pipeline, which extracts frequency-domain features from the acquired signals and estimates the circuit topology and its component values; and (iii) a PC application, used both for live visualization of the measured spectrum and for building the labeled dataset used to train the ML models offline.

The microcontroller has two modes:

- **Inference mode:** the extracted feature vector (Magnitude, Phase) is passed directly to the on-device Random Forest classifier and CNN regressor cascade, and the resulting topology and component estimates are output on-device.
- **Data collection mode:** the extracted feature vector is instead streamed over USART to the PC application, which renders it as live Bode/Nyquist plots and logs it, together with automated labeling, creating the training dataset.

Both modes share the same analog front-end and the same on-device AC signal analysis stage; only the destination of the resulting feature vector differs.

The dataset collected through the PC application is used entirely offline to train the two-stage cascade the Random Forest topology classifier and the four topology-specific 1D-CNN parameter regressors (Section 2.6) following the training and validation procedure described in Section 3.3.

Once trained, the classifier, the four regressors, and their associated scalers, label encoder, and configuration parameters are converted by a custom Python export script into a single dependency-free C header file.

This header file is then re-deployed onto the microcontroller (dashed arrow in Figure 6), closing the loop between data collection and on-device inference. In inference mode, the same feature vector that would otherwise be streamed to the PC application is instead passed directly to this on-device cascade, so that no separate signal-processing path is needed between the two operating modes.

Figure 6 shows the complete data flow across all three subsystems.

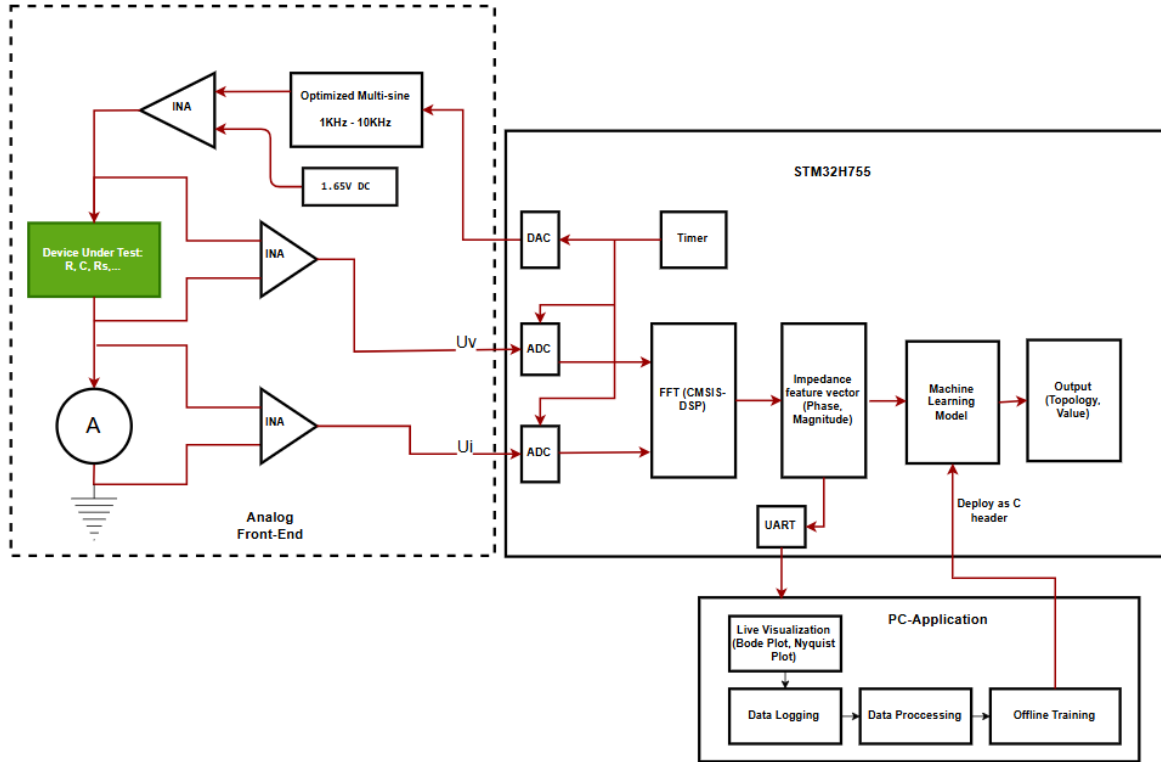


Figure 6: Overall proposed system architecture.

The design choices follow directly from the evaluation against the trade-offs discussed theoretically in Chapter 1. Specifically:

**Measurement method.** Among the measurement principles reviewed in Section 1.4, the I–V method was adopted because it interfaces directly with the DAC/ADC peripherals already integrated on the STM32, without requiring any circuitry (Section 1.4). Within the I–V method, digital AC signal processing was chosen over analog demodulation because the excitation is multi-frequency; analog demodulation would require one dedicated demodulator per excited frequency, which does not scale (Section 1.4.2).

**Excitation signal.** A multisine excitation signal is adopted over a sequential sine sweep to minimize total acquisition time and satisfy system stability requirements. The frequency range of 1 KHz to 10 kHz is tailored for bio-impedance applications (as illustrated in Figure 2). To prevent signal clipping within the fixed voltage ranges of the microcontroller’s DAC and ADC, crest factor optimization is essential. Adapting the hybrid stochastic-deterministic optimization approach proposed by [6], the multisine initial phases are optimized using an interleaved sigmoid transform and Gauss-Newton algorithm to achieve a low crest factor and high signal-to-noise ratio efficiently.

**Synchronization strategy.** To avoid the spectral leakage described in Section 1.6, the digital-to-analog converter (DAC) generating the multi-sine signal and the two analog-to-digital converters (ADCs) measuring the voltage and current across the device under test (DUT) are all driven synchronously by a shared clock source via a timer-triggered mechanism.

**AC Analysis.** Once the voltage and current signals across the DUT are synchronously acquired, the Fast Fourier Transform (FFT) is applied to extract their amplitude and phase. To meet real-time constraints on resource-limited microcontrollers, the software leverages the

CMSIS-DSP library, utilizing optimized SIMD instructions and hardware acceleration units (such as single-cycle MAC on ARM Cortex-M cores) to accelerate processing speed and minimize CPU cycle consumption.

**Dual-mode operation and the role of the PC application.** Because dataset construction and on-device inference both require the same frequency-domain feature vector, the system was designed so that the AC-analysis stage feeds a single shared output, which is then routed either to the on-device ML cascade or to the PC application depending on the selected operating mode. This avoids maintaining two separate signal-processing paths for measurement and dataset-collection purposes, and lets the operator visually verify each spectrum (Bode/Nyquist) before it is committed to the training dataset (Section 2.5).

**Machine learning architecture.** The two-stage cascade (topology classification followed by per-topology parameter regression) reflects the decoupled roles introduced conceptually in Sections 2.6. Random Forest was selected for Stage 1 on the theoretical grounds of its robustness on discrete, threshold-separable classes and its insensitivity to feature scale, this was subsequently confirmed empirically, as Random Forest outperformed the Decision Tree, SVM (RBF), and MLP baselines evaluated for comparison. A 1D-CNN was selected for Stage 2 because the regression target (R and/or C) is a continuous function of a structured frequency sequence.

**Embedded ML deployment.** Among the deployment options compared in Section 1.10 (Table 4), a custom C code-generation approach was selected, as it was the only option supporting the exact two-stage cascade architecture (Random Forest + independent per-class CNNs) without restructuring the model into a single network graph, at the cost of forgoing vendor-provided quantization and hardware-optimized kernels.

## 2.2 Hardware Setup

### 2.2.1 Circuit Design

The hardware front-end for the impedance spectroscopy measurement system is designed to generate a precisely conditioned excitation signal, apply it to the target load, and capture the resulting differential voltage and current. Two active components dominate the signal chain: the AD8421 instrumentation amplifier, used in each of the three signal-conditioning stages (U4, U1, U2), and the ADA4522-1 zero-drift operational amplifier (U6). The AD8421 was selected for its  $3\text{ nV}/\sqrt{\text{Hz}}$  input voltage noise, single-resistor gain programmability ( $G = 1 + 9.9\text{ k}\Omega/R_G$ ), and high common-mode rejection ratio (94 dB minimum at  $G = 1$ ) [7], all of which are essential for rejecting common-mode interference on the differential  $U_v$  and  $U_i$  measurement paths at frequencies up to the 10 kHz upper bound of the excitation spectrum. The ADA4522-1 was chosen for the DC reference buffer specifically for its ultralow offset voltage ( $5\text{ }\mu\text{V}$  maximum) and near-zero offset drift ( $22\text{ nV}/^\circ\text{C}$  maximum)[8]; this ensures that the 1.65 V virtual ground remains stable with temperature and supply variation, preventing a time-varying DC bias from being injected into the excitation and measurement paths. To facilitate a clear understanding of the signal flow, the circuit is logically partitioned into five sequential stages, which is also depicted here in figure 8:

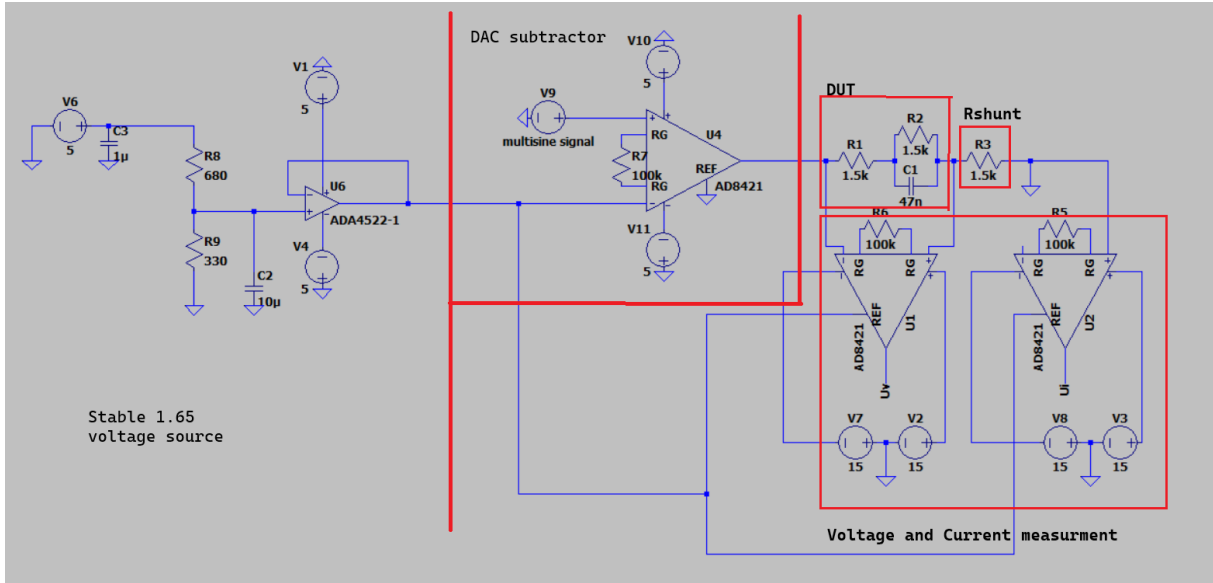


Figure 7: Overall circuit in LTSpice

### Physical realization of the analog front-end and measurement system.

The prototype was assembled on stacked perfboards, hand-wired point-to-point to implement the DC reference generation, DAC subtractor, DUT interface, and the two AD8421-based instrumentation amplifier stages for voltage and current sensing (Figure 8). The STM32H755ZI-Q Nucleo-144 board (bottom) provides the DAC excitation and dual-ADC acquisition, interfacing with the analog front-end via the wired connections shown. While the dense wiring reflects the iterative, breadboard-style prototyping process, the corresponding LTSpice schematic (Figure 7) gives the definitive circuit topology and connectivity.

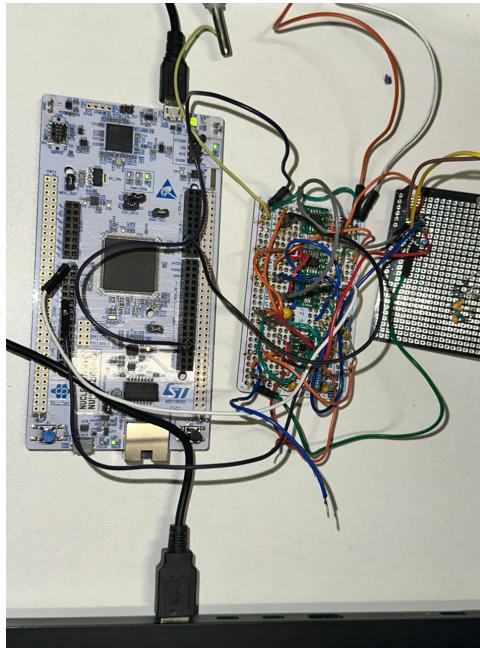


Figure 8: Real circuit

1. **Stage 1: DC Reference Generation:** The system begins by establishing a highly stable 1.65V DC reference voltage. This bias is derived from the main 5V supply and buffered to provide a low-impedance foundation. This ensures that the single-supply Analog-to-Digital

Converters (ADCs) and instrumentation amplifiers can operate around a centered, virtual ground.

The analog front-end also requires a symmetric  $\pm 5\text{V}$  supply for the amplifier stages. During bench testing, this was provided by a HAMEG HM7042 triple power supply (Figure 9), with two channels set to  $5.0\text{V}$  to form the  $+5\text{V}$  and  $-5\text{V}$  rails.

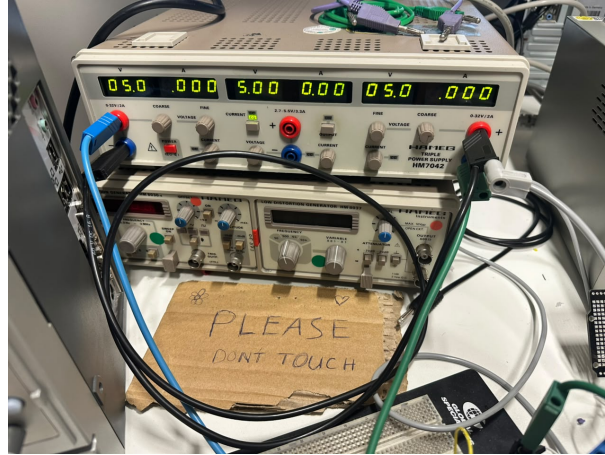


Figure 9: HAMEG HM7042 triple power supply configured to generate the  $\pm 5\text{V}$  analog rails during bench testing.

2. **Stage 2: Signal Conditioning (DAC Output):** A unipolar multisine excitation signal (ranging from  $0.0\text{V}$  to  $3.3\text{V}$ ) is generated by the STM32 microcontroller. This stage utilizes an instrumentation amplifier to subtract the  $1.65\text{V}$  reference from the source, effectively level-shifting the waveform into a bipolar AC signal ( $-1.65\text{V}$  to  $+1.65\text{V}$ ). This critical step prevents DC polarization of the target load.
3. **Stage 3: The Device Under Test (DUT):** The conditioned AC signal is driven into the measurement path. This physical path consists of the target impedance network (the DUT) connected in series with a known precision shunt resistor to ground.
4. **Stage 4: Differential Voltage Measurement ( $U_v$ ):** A dedicated instrumentation amplifier is positioned directly across the DUT to measure the precise voltage drop. This differential AC signal is captured and level-shifted back to a  $1.65\text{V}$  DC center to match the unipolar input requirements of the microcontroller's ADC.
5. **Stage 5: Current Measurement ( $U_i$ ):** A parallel instrumentation amplifier measures the voltage drop across the series shunt resistor. Because the shunt resistance is a known constant, this voltage is directly proportional to the current flowing through the DUT. Like the voltage measurement, this signal is level-shifted and filtered before being routed to the ADC.

By capturing both the  $U_v$  and  $U_i$  signals simultaneously through these five stages, the microcontroller can perform Fast Fourier transforms algorithm on the synchronized data to calculate the complex impedance of the DUT across the generated frequency spectrum.

### 2.2.2 Stable $1.65\text{V}$ Reference Supply

A stable reference voltage (nominally  $1.65\text{V}$ ) is required to bias the Digital-to-Analog Converter (DAC) and Analog-to-Digital Converter (ADC) components without introducing drift. This is achieved using a voltage divider network connected to a stable  $5\text{V}$  source. To improve power

supply rejection and ensure stability, the 5 V source is initially decoupled with a 1  $\mu$ F capacitor (C3).

The voltage divider itself consists of a 680  $\Omega$  resistor ( $R_8$ ) and a 330  $\Omega$  resistor ( $R_9$ ), which scales the voltage according to the standard voltage divider formula:

$$V_{ref} = 5 \text{ V} \times \frac{330 \Omega}{680 \Omega + 330 \Omega} \approx 1.634 \text{ V} \quad (17)$$

This scaled voltage, which provides an effective nominal center point for the 3.3 V ADC logic, is subsequently filtered through a 10  $\mu$ F capacitor (C2) to attenuate high-frequency noise. Finally, the signal is buffered using an ADA4522-1 operational amplifier (U6) configured as a unity-gain voltage follower. This configuration provides a low-impedance output for the reference voltage, ensuring stability under varying load conditions from the downstream components [8].

### 2.2.3 The DAC Output Stage

The DAC partition utilizes an AD8421 instrumentation amplifier (U4) to condition the multisine signal generated by the STM32 microcontroller. The AD8421 is selected for its high common-mode rejection and differential input configuration [7].

The STM32 generates a unipolar multisine signal ( $V_{dac}$ ) ranging from 0.0 V to 3.3 V. To prevent DC polarization in the subsequent ADC stage and the Device Under Test (DUT), this signal must be centered around zero. The AD8421 achieves this by subtracting the buffered reference voltage, mentioned in Sec.2.2.2, from the source signal.

To appropriately scale the output, this stage utilizes a gain resistor ( $R_7$ ) of 100 k $\Omega$  connected across the RG pins. According to the AD8421 datasheet [7], the gain  $G$  is determined by the equation  $G = 1 + \frac{9.9 \text{ k}\Omega}{R_G}$ , resulting in a nominal gain of approximately 1.099. Because  $R_G \gg 9.9 \text{ k}\Omega$ , the circuit operates close to the unity-gain condition; in this regime, the AD8421 datasheet specifies its widest bandwidth (10 MHz,  $G = 1$ ) and highest slew rate (35 V/ $\mu$ s) [7], both of which comfortably exceed the requirements of the 1 KHz–10 kHz multisine excitation band and preserve the phase fidelity of the injected signal.

The general transfer function for the AD8421 is explicitly defined in its datasheet as [7]:

$$V_{OUT} = G \times (V_{+IN} - V_{-IN}) + V_{REF} \quad (18)$$

In this circuit configuration, the multisine signal is fed into the non-inverting input ( $V_{+IN} = V_{dac}$ ), the stable reference voltage is connected to the inverting input ( $V_{-IN} \approx 1.634 \text{ V}$ ), and the output reference pin is tied directly to ground ( $V_{REF} = 0 \text{ V}$ ). Substituting these parameters yields the specific output equation:

$$V_{OUT} = 1.099 \times (V_{dac} - 1.634 \text{ V}) \quad (19)$$

This mathematical shift effectively translates and slightly amplifies the original unipolar signal into a bipolar AC waveform.

The physical pin configuration of the AD8421 in this setup is illustrated in Figure 10 and detailed as follows:

- **Pin 1 (-IN):** Connected to the stable reference supply from U6.
- **Pins 2 & 3 (RG):** Connected to the 100 k $\Omega$  gain resistor ( $R_7$ ).
- **Pin 4 (+IN):** Connected to the multisine signal input from the STM32.
- **Pin 5 (-VS):** Connected to the  $-5 \text{ V}$  power supply.
- **Pin 6 (REF):** Connected to system ground (GND).

- **Pin 7 (VOUT):** The conditioned output signal, passed to the DUT.
- **Pin 8 (+VS):** Connected to the +5 V power supply.

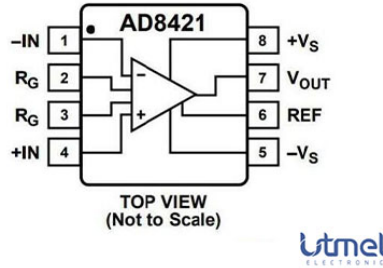


Figure 10: Pin layout and configuration of the AD8421

1

### 2.2.4 The Device Under Test (DUT)

To validate the measurement system with a known ground truth, the first Device Under Test (DUT) is a passive  $RC$  network chosen to emulate the electrical behavior of a typical electrochemical or dielectric load, rather than a purely resistive element. As shown in the LTSpice schematic, the network consists of a  $1.5\text{ k}\Omega$  resistor ( $R_1$ ) connected in series with a parallel combination of a  $47\text{ nF}$  capacitor ( $C_1$ ) and a second  $1.5\text{ k}\Omega$  resistor ( $R_2$ ). This topology produces an impedance magnitude and phase that vary predictably across the  $1\text{ kHz}$  to  $10\text{ kHz}$  excitation band: at low frequency,  $C_1$  presents a high reactance and the network impedance approaches  $R_1 + R_2 = 3\text{ k}\Omega$ , whereas at high frequency  $C_1$  increasingly shorts  $R_2$ , driving the impedance toward  $R_1 = 1.5\text{ k}\Omega$ . The characteristic corner frequency of this transition is set by the parallel combination,  $f_c = \frac{1}{2\pi R_2 C_1} \approx 2.26\text{ kHz}$ , placing it within the excitation band so that both the resistive and capacitive regimes of the network can be exercised and resolved by the FFT analysis in later chapters.

To enable current measurement, a  $1.5\text{ k}\Omega$  shunt resistor ( $R_3$ ) is connected in series with the DUT and tied to system ground. Because  $R_3$  is placed after the DUT rather than before it, the current flowing through the entire branch—and therefore through the DUT—is sensed directly, avoiding the need to correct for any bias current drawn by the U1/U2 amplifier inputs. When the conditioned multisine signal from the DAC stage ( $V_{OUT}$  of U4) is applied across this series combination, the resulting voltage across the DUT and the voltage across  $R_3$  are simultaneously captured by the U1 and U2 instrumentation amplifiers, respectively, for impedance spectroscopy analysis.

### 2.2.5 The ADC Front-End (Signal Measurement)

To accurately capture the voltage and current for the microcontroller's Analog-to-Digital Converter (ADC), the measurement stage employs two AD8421 instrumentation amplifiers. These amplifiers provide the necessary high common-mode rejection, low noise, and precise differential sensing required for reliable impedance calculations [7].

- **Voltage Measurement ( $U_v$ ):** The first AD8421 (U1) is configured to measure the differential voltage directly across the DUT. The non-inverting input (+IN) is connected to the node between the DUT and the shunt resistor, while the inverting input (-IN) is connected to the node preceding the DUT.

- **Current Measurement ( $U_i$ ):** The second AD8421 (U2) is dedicated to current measurement by capturing the voltage drop across the  $1.5\text{ k}\Omega$  shunt resistor. Its non-inverting input (+IN) is tied directly to system ground (GND), and its inverting input (-IN) is connected to the node between the DUT and the shunt resistor.

Both amplifiers utilize a gain resistor ( $R_G$ ) of  $100\text{ k}\Omega$ . According to the AD8421 transfer function ( $G = 1 + 9.9\text{ k}\Omega/R_G$ ), this results in a nominal gain of approximately 1.099, matching the gain used in the DAC conditioning stage so that  $U_v$  and  $U_i$  share a consistent scale factor when the complex impedance  $Z = U_v/U_i \times R_{shunt}$  is computed in firmware.

Furthermore, to ensure the measured bipolar signals are compatible with the unipolar  $0.0\text{ V}$  to  $3.3\text{ V}$  input range of the STM32's ADC, the reference pins (REF) of both AD8421 amplifiers are tied to the stable  $1.65\text{ V}$  supply described in the previous section. This effectively level-shifts the  $U_v$  and  $U_i$  AC outputs to a  $1.65\text{ V}$  DC center point.

## 2.3 Firmware Implementation

The central processing and control unit of the measurement system is the STM32H755ZI-Q microcontroller, housed on a Nucleo-144 development board [9].

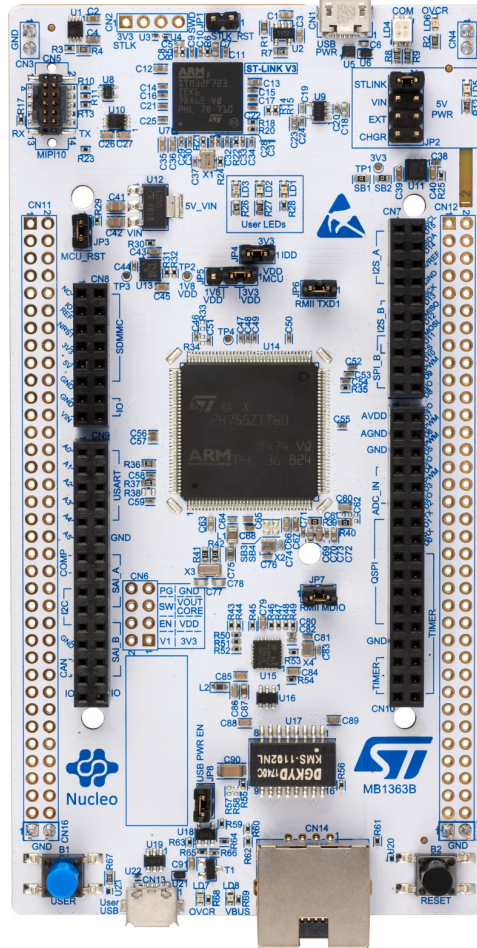


Figure 11: STM32 Nucleo-144 development board

To ensure strict timing deterministic behavior—which is critical for accurate phase measurements in impedance spectroscopy—the firmware heavily leverages hardware timers and Direct Memory Access (DMA). The code flow correlates directly to the hardware partitions as follows:

- **Signal Generation (DAC & DMA):** A 4096-point multisine waveform is pre-calculated and stored in the microcontroller's memory. Timer 4 (TIM4) is configured to trigger the Digital-to-Analog Converter (DAC1), while a DMA channel continuously streams the waveform array to the DAC register. This allows the STM32 to output the unipolar excitation signal to the hardware subtractor stage continuously without utilizing CPU cycles.
- **Simultaneous Acquisition (ADCs & DMA):** The differential voltage ( $U_v$ ) and current ( $U_i$ ) signals conditioned by the AD8421 front-end are routed to two separate Analog-to-Digital Converters (ADC1 and ADC2). Timer 2 (TIM2) determines the sampling rate of both ADCs, making them synchronize in time and reducing the spectral leakage problem.

Two dedicated DMAs simultaneously streams ADC samples in data registers to memory buffers ( `AdcBuffer1` and `AdcBuffer2` ) until completing 4096 samples for each channel.

- **Interrupt and DSP Execution:** Once the DMA completes filling the 4096-point buffers, a hardware interrupt ( `HAL_ADC_ConvCpltCallback` ) will be called. Inside this callback function, we set a completion flag. The main loop halts the acquisition timer and begins processing the buffers. Using the CMSIS-DSP library, the firmware applies the Fast Fourier Transform algorithm to the sampled data. This extracts the precise magnitude and phase shift for the specific frequency bins present in the multisine signal.
- **Data Transmission:** After calculating the accumulated phase and magnitude, the results are formatted and transmitted via USART3 to a connected PC for final logging and visualization.

The sequential logic of the firmware, from peripheral initialization to the continuous DSP loop, is illustrated in Figure 12.

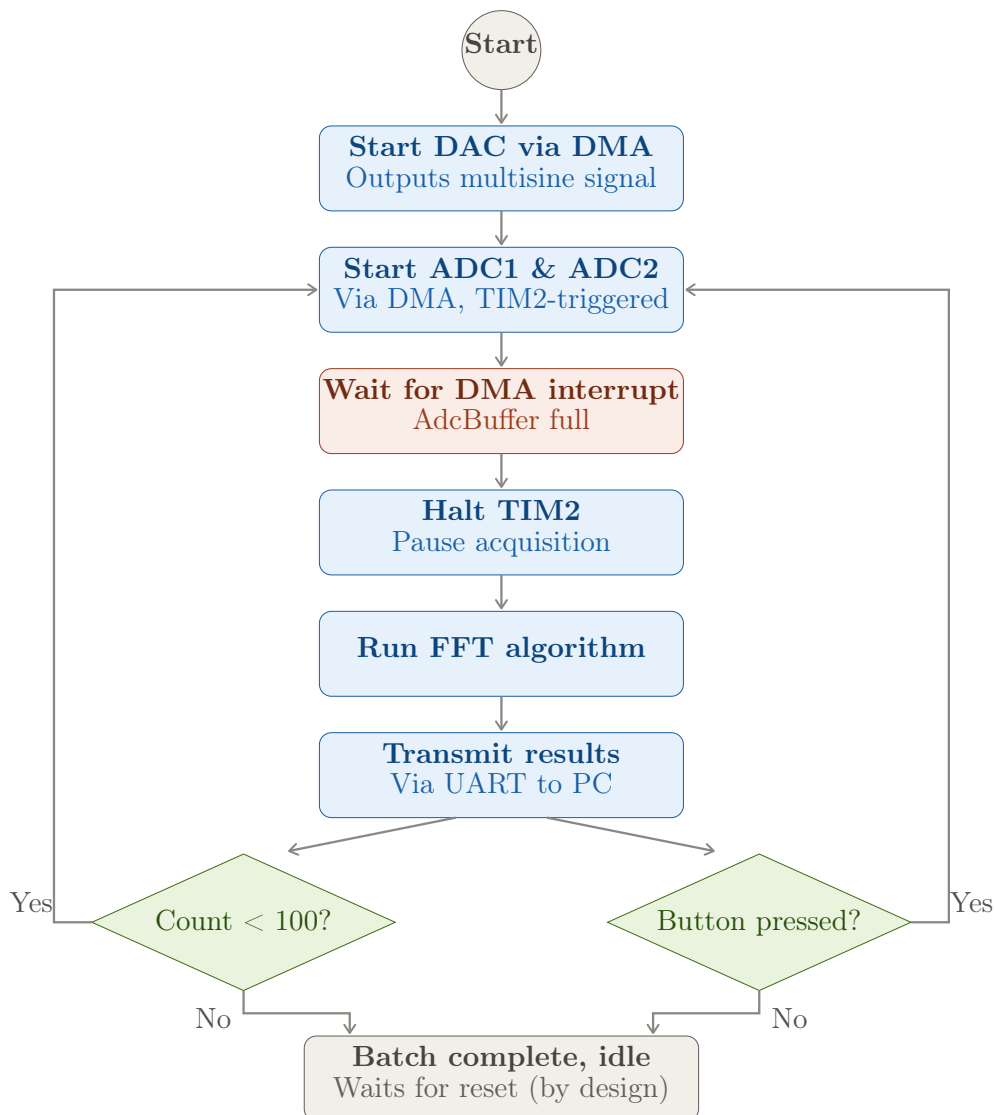


Figure 12: STM32 execution flow chart

The firmware follows a classic *initialize-then-loop* architecture typical of STM32 HAL-based projects, structured around DMA-driven peripheral transfers to minimize CPU overhead during signal acquisition.

**Initialization phase.**

After the HAL and system clocks are configured (`SystemClock_Config`, `PeriphCommonClock_Config`), the required peripherals are initialized: GPIO, DMA, the two ADC instances (`ADC1`, `ADC2`), USART3 for serial reporting, DAC1 for waveform generation, and the timers TIM2/TIM4 used to trigger ADC sampling and DAC output, respectively. The DAC is started first, in circular DMA mode, continuously outputting the precomputed 4096-sample multisine excitation signal. Both ADC channels are then calibrated (offset, single-ended mode) before their DMA transfers and the sampling timer (TIM2) are armed, so that `AdcBuffer1` and `AdcBuffer2` are filled synchronously.

**Acquisition and processing loop.**

The main `while(1)` loop is interrupt-driven: it remains idle until `HAL_ADC_ConvCpltCallback` sets the `DmaCpltFlag1` flag, signaling that a full 4096-sample acquisition on both channels has completed. Once triggered, TIM2 is stopped and the flag is cleared. Each channel buffer is converted into a complex-valued array (real samples with zero-valued imaginary parts) and processed by the FFT and bin-extraction stage described above, yielding the impedance spectrum at the 15 excited frequencies for this acquisition cycle.

**Restarting the next acquisition.**

After each processed acquisition, the firmware checks two independent conditions to decide whether to start ADC1/ADC2 and TIM2 again for a new measurement cycle. First, if the run counter has not yet reached the configured limit of 100 iterations, acquisition restarts automatically. Second, regardless of the run counter, a debounced user-button interrupt (EXTI) can also trigger a restart, letting the user manually begin a new measurement from the front panel at any time. Once the run counter reaches 100 and no button press is pending, neither condition holds: the ADC/TIM2 are not restarted, the green LED is turned on to indicate the batch is complete, and the system remains idle until reset. This 100-iteration batch is the same acquisition loop later reused during embedded deployment (Section 2.7.2) to obtain the majority-vote and averaged inference result from repeated measurements. Note that this listing corresponds to the *data-collection-mode* firmware, which streams the extracted feature vector to the PC application (Section 2.1); the deployed inference-mode firmware replaces the UART transmission step with a call into the on-device Random Forest + 1D-CNN cascade.

## 2.4 Graphical User Interface for Real-Time EIS Visualization

### 2.4.1 Purpose and Role of the GUI

A graphical user interface (GUI) was developed as the host-side application of the project. Its main purpose is to receive measurement data transmitted by the STM32 through a serial connection, visualize the impedance response in real time, and store the measured data for subsequent analysis.

The GUI is not only a presentation tool. It is an important verification and debugging interface between the embedded acquisition system, the analog front-end, and the machine learning stage. Before using the collected data for classification or regression, the measured impedance spectra must be checked for physical plausibility, repeatability, noise, distortion, and measurement errors [10, 11]. Therefore, the GUI provides a direct way to inspect the quality of the measured EIS data.

The overall role of the GUI in the system can be summarized as follows:

1. Establish a serial connection between the PC and the STM32-based measurement system.
2. Receive and parse real-time measurement frames transmitted through UART.

3. Detect whether the incoming data correspond to time-domain signals or an impedance sweep.
4. Display the EIS response in multiple graphical formats, including Nyquist and Bode plots[12].
5. Compare repeated impedance sweeps to evaluate repeatability and measurement stability[13].
6. Support debugging of the analog front-end, excitation source, ADC acquisition, DSP calculations, and serial communication.
7. Store the acquired measurement data for offline analysis, dataset construction, machine learning training, and regression.

### 2.4.2 Serial Communication and Data Acquisition

The GUI communicates with the embedded system through a configurable serial port. The user can select the COM port and baud rate directly from the interface. The default communication speed is 115 200 baud, while the GUI supports a wider baud-rate range for testing and future extensions [14].

A function `SerialReaderThread` is used to read UART data continuously in the background [14]. This architecture ensures that the UI remains responsive while measurements are received and plotted [15]. The raw serial lines are also displayed in a console panel, which is useful for checking the communication protocol and detecting malformed or unexpected messages.

A new impedance sweep is detected when the frequency index returns to zero. The received points are collected into one complete sweep and transferred to the visualization module. This allows the GUI to update the plots sweep by sweep rather than plotting isolated data points.

For the current measurement setup, the frequency index is mapped to a logarithmic frequency range from approximately 10 Hz to 100 kHz. A logarithmic frequency scale is appropriate because EIS measurements typically cover multiple decades of frequency [12, 16].

### 2.4.3 Graphical Interface Design

The GUI uses a two-panel layout, as illustrated in Figure 13. The left panel contains controls for serial communication, plot selection, data management, measurement counters, and the serial console. The right panel is reserved for the active visualization area.

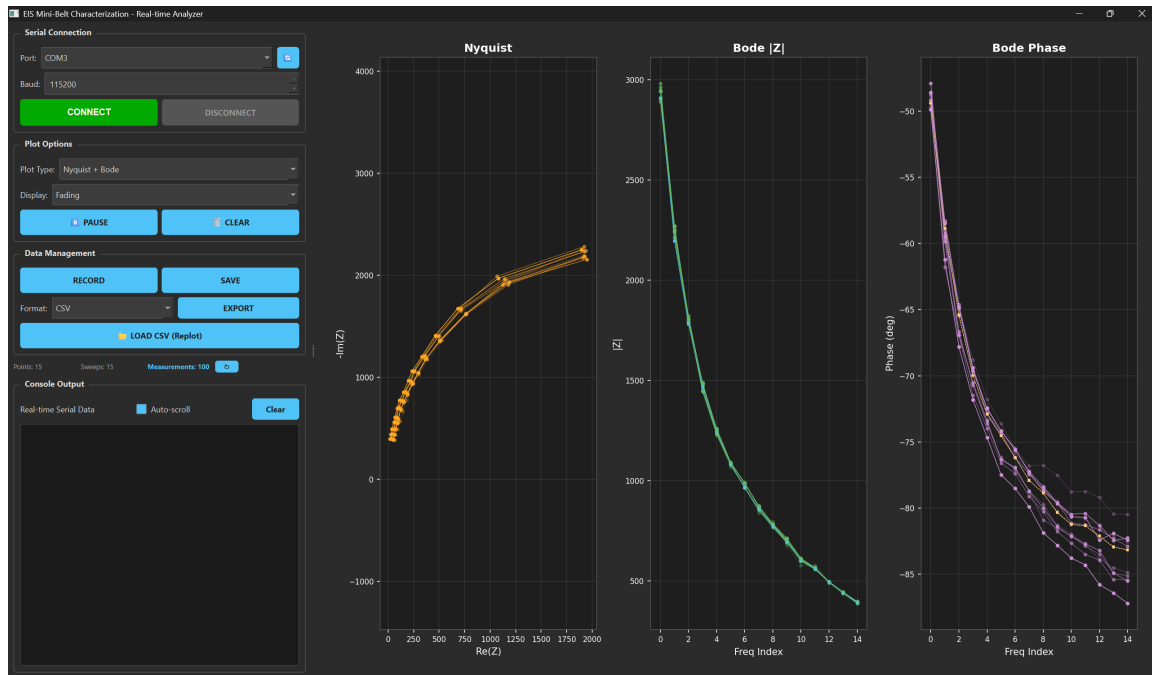


Figure 13: Overview of the developed real-time EIS graphical user interface.

The main GUI functions are summarized in Table 5.

Table 5: Main functions provided by the GUI.

| Function                        | Description                                                                                                                    |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Serial connection</b>        | Selects the COM port and baud rate, then connects or disconnects the PC from the STM32 device.                                 |
| <b>Real-time console</b>        | Displays received serial messages and status information for protocol verification and debugging.                              |
| <b>Mode detection</b>           | Automatically distinguishes between time-domain signal frames and impedance sweep data.                                        |
| <b>Plot selection</b>           | Allows selection of Nyquist, Bode magnitude, Bode phase, real impedance, imaginary impedance, combined, and time-domain views. |
| <b>Real-time display</b>        | Updates the selected plot periodically while new sweeps are received.                                                          |
| <b>Fading display</b>           | Shows several recent sweeps with decreasing visibility, which helps evaluate repeatability and drift over time.                |
| <b>Pause and clear</b>          | Pauses plot updates for inspection and clears the stored measurement buffers when a new experiment starts.                     |
| <b>Data recording</b>           | Records raw serial data with timestamps into a CSV file.                                                                       |
| <b>Buffer saving and export</b> | Saves buffered impedance sweeps for offline processing and dataset generation.                                                 |

### 2.4.4 Nyquist Plot

The Nyquist plot is one of the most important representations in impedance spectroscopy [12, 17]. It can visualize the relationship between the real and imaginary parts of the complex impedance:

$$Z(f) = \Re\{Z(f)\} + j\Im\{Z(f)\}. \quad (20)$$

Conventionally, the Nyquist plot displays  $\Re\{Z\}$  on the horizontal axis and  $-\Im\{Z\}$  on the vertical axis [17]. Each point represents the impedance at one excitation frequency. The frequency itself is not directly plotted, but the trajectory across the complex plane reveals the electrical behavior of the device under test.

For a resistor, the impedance is approximately real and nearly independent of frequency:

$$Z_R = R. \quad (21)$$

Therefore, an ideal resistor appears close to a point on the real axis in the Nyquist plot. Deviations from this behavior may indicate parasitic capacitance, inductance, measurement noise, ADC errors, insufficient settling time, or limitations of the analog front-end.

For an ideal capacitor,

$$Z_C = \frac{1}{j\omega C}, \quad (22)$$

where  $\omega = 2\pi f$ . The impedance has a negative imaginary component and its magnitude decreases as the frequency increases.

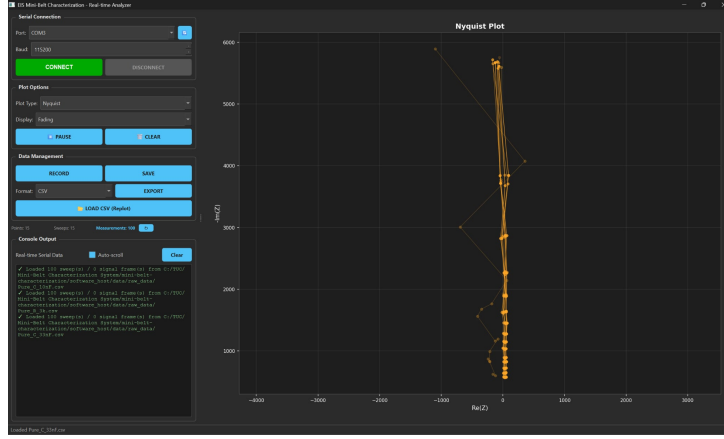
For a series RC circuit,

$$Z_{\text{series}} = R + \frac{1}{j\omega C}, \quad (23)$$

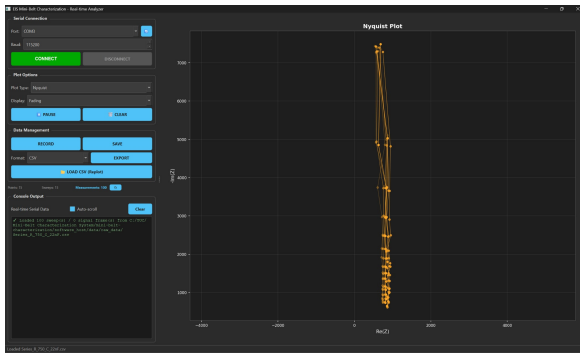
while for a parallel RC circuit,

$$Z_{\text{parallel}} = \left( \frac{1}{R} + j\omega C \right)^{-1}. \quad (24)$$

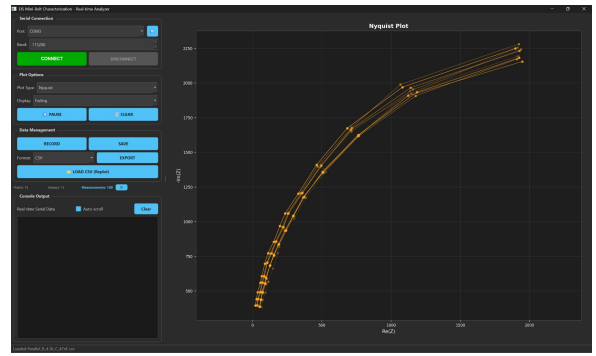
These different circuit topologies produce different Nyquist trajectories [16]. Consequently, the Nyquist plot can be used to identify whether the measured response is consistent with the expected resistor, capacitor, series RC, or parallel RC behavior.



(a) Capacitor



(b) Series RC



(c) Parallel RC

Figure 14: Example Nyquist plots for the investigated circuit topologies.

The Nyquist plot is especially useful during debugging. For example, unexpected loops, strongly scattered points, discontinuities, or non-physical trajectories can reveal problems in the acquisition chain [18, 19]. Such problems may originate from the excitation signal, current-to-voltage conversion, synchronization between DAC and ADC, FFT processing, incorrect phase calculation, communication errors, or insufficient averaging.

## 2.4.5 Bode Plots

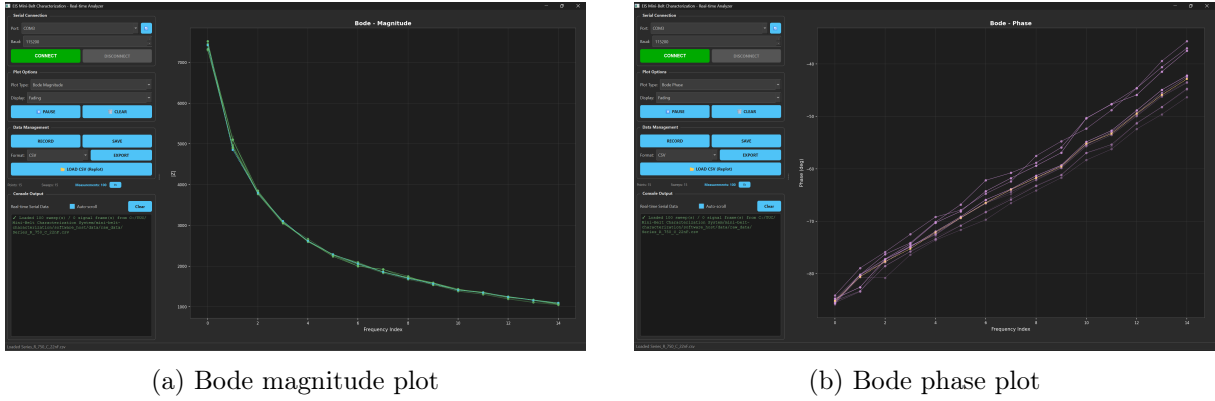
The GUI also provides Bode magnitude and Bode phase plots. In these plots, the impedance magnitude and phase are displayed as functions of frequency:

$$|Z(f)| = \sqrt{\Re\{Z(f)\}^2 + \Im\{Z(f)\}^2}, \quad (25)$$

$$\varphi(f) = \tan^{-1} \left( \frac{\Im\{Z(f)\}}{\Re\{Z(f)\}} \right). \quad (26)$$

The Bode magnitude plot typically uses a logarithmic frequency axis [16]. It allows the user to inspect how the impedance magnitude changes over the measurement range. The Bode phase plot shows the phase shift between voltage and current, which is particularly important for distinguishing resistive and capacitive behavior.

For an ideal resistor, the magnitude remains approximately constant and the phase is close to  $0^\circ$ . For an ideal capacitor, the magnitude decreases with frequency and the phase approaches  $-90^\circ$  [17, 16]. Series RC and parallel RC circuits exhibit transitions between resistive and capacitive regions, depending on the selected component values and the measured frequency range.

Figure 15: Bode plots of Series:  $R = 750\Omega$ ,  $C = 22\text{nF}$ 

The Bode plots provide a practical way to evaluate the measurement system. A noisy magnitude curve, an unstable phase curve, missing frequency regions, or unrealistic phase jumps can indicate problems in the hardware or firmware [19]. By comparing the plots with the theoretical behavior of known  $R$ ,  $C$ , series RC, and parallel RC circuits, the user can identify weak parts of the measurement chain and improve the system before collecting the final dataset.

#### 2.4.6 Additional Visualization Modes

In addition to Nyquist and Bode plots, the GUI supports the following views:

- **Real part versus frequency:** displays  $\Re\{Z\}$  as a function of frequency.
- **Imaginary part versus frequency:** displays  $\Im\{Z\}$  as a function of frequency.
- **Nyquist plus Bode view:** combines the Nyquist plot with magnitude and phase plots in one view.
- **Nyquist plus real/imaginary view:** combines the complex-plane representation with the individual real and imaginary components.

These multiple views make it possible to inspect the same sweep from different perspectives. For example, a Nyquist plot may reveal a non-physical trajectory, while the Bode phase plot can indicate at which frequency the phase error begins. Similarly, the time-domain plot can help determine whether the issue originates before the FFT and impedance calculation stages.

#### 2.4.7 Use of the GUI for Debugging and Validation

The GUI is an essential tool for validating the entire EIS measurement chain. The measurement path includes the multisines excitation source, the device under test, the analog front-end, the ADC channels, the STM32 DSP processing, and UART communication. Errors introduced at any of these stages can affect the final impedance spectrum.

The GUI supports debugging in the following ways:

1. **Verification of circuit behavior:** Known resistor, capacitor, series RC, and parallel RC circuits can be measured and compared with their expected Nyquist and Bode characteristics [20, 21].
2. **Detection of hardware limitations:** Excessive noise, distorted curves, offset errors, unexpected phase shifts, or unstable high-frequency points can indicate limitations in the analog front-end, wiring, grounding, shielding, or ADC resolution [19].

3. **Verification of firmware processing:** The plots can reveal incorrect FFT results, incorrect magnitude or phase calculations, frequency-index errors, or synchronization problems between DAC excitation and ADC sampling.
4. **Verification of UART communication:** The real-time serial console exposes the raw frames received from the STM32. This helps verify the message format and detect malformed, missing, or interrupted data.
5. **Repeatability analysis:** The fading mode overlays recent sweeps. If repeated sweeps differ strongly, the user can investigate drift, noise, unstable excitation amplitude, poor contacts, or insufficient averaging [13, 18].

Therefore, the GUI transforms the system from a simple data logger into an interactive measurement and debugging environment. It allows incorrect or low-quality measurements to be identified before they enter the machine learning pipeline.

## 2.4.8 Data Logging and Relation to Machine Learning

The GUI provides recording, buffer saving, and export functions. Raw serial messages can be recorded with timestamps, while complete impedance sweeps can be saved to CSV file [22]. These functions are important because the machine learning and regression stages require a reliable, labeled, and physically meaningful dataset.

The expected workflow is:

1. Connect a known test circuit to the measurement system.
2. Use the GUI to inspect the Nyquist and Bode plots.
3. Verify that the measured response is consistent with the expected circuit topology.
4. Repeat the measurement until the response is stable and repeatable.
5. Save the validated impedance sweep to the dataset.
6. Assign the correct topology label and component values.
7. Use the validated dataset for topology classification and parameter regression.

This validation step is important because machine learning models cannot compensate for fundamentally incorrect input data. If the hardware, firmware, or signal processing produces noisy or biased impedance spectra, the classifier and regressor may learn measurement artifacts instead of the actual electrical behavior of the circuit.

The GUI therefore acts as a quality-control layer before the machine learning stage [11]. It helps ensure that the data used for training represent meaningful impedance characteristics of resistor, capacitor, series RC, and parallel RC circuits. This improves the reliability of the later topology classifier and the regression models used to estimate  $R$  and  $C$  values.

## 2.5 Data Acquisition and Dataset Preparation

### 2.5.1 Dataset Construction from Raw Data Streams

At first with the raw data response by STM32, we extracting labels and physical parameters from measurement filenames:

- The `parse_filename` function extracts the `Label`, `Value_R`, and `Value_C` from filename patterns such as `Pure_C`, `Pure_R`, `Series`, and `Parallel`. This approach ensures that every measurement record is associated with a topology label and the actual component values (ground truth) for learning [23].
- **Before processing:** Filenames are formatted as `Parallel_R_1.3k_C_100nF.csv` or `Pure_C_100nF.csv`.
- **After processing:** Data is extracted into discrete parameters.

Table 6: Examples of filename parsing into structured parameters.

| Original Filename (Mags and Phases)      | Label (Topology)         | Value_R ( $\Omega$ ) | Value_C (nF) |
|------------------------------------------|--------------------------|----------------------|--------------|
| <code>Parallel_R_1.3k_C_100nF.csv</code> | <code>Parallel_RC</code> | 1300.0               | 100.0        |
| <code>Series_R_3k_C_33nF.csv</code>      | <code>Series_RC</code>   | 3.0                  | 33.0         |
| <code>Pure_C_100nF.csv</code>            | <code>Capacitor</code>   | 0.0                  | 100.0        |
| <code>nothing.csv</code>                 | <code>nothing</code>     | 0.0                  | 0.0          |

### 2.5.2 Cleaning and Splitting Measurement Data Frames

Each row of measurement data is parsed into `Index`, `Magnitude`, `Phase`, `AccMagnitude`, and `AccPhase`. Subsequently, `Label`, `Value_R`, and `Value_C` are appended to each sample.

- **Before processing:** Data is contained within a single `data` column formatted as a comma-separated string, accompanied by a timestamp. For example:

```
timestamp, "0, 139435.593, 175.297, ..."
```

- **Processing method:** Null values (NaN) are removed, and valid data rows starting with a digit are filtered. The string is then split into independent physical feature columns. Timestamps are discarded, as they hold no significance in the frequency domain [23].
- **After processing:** Each data point is structured into a table with the following columns: `Index` (frequency measurement sequence number), `Magnitude`, `Phase`, `Label`, `Value_R`, and `Value_C`.

Table 7: Example rows of the 1 file processed dataset before flattening.

| Index | Magnitude | Phase   | Label       | Value_R | Value_C |
|-------|-----------|---------|-------------|---------|---------|
| 0     | 1268.305  | -19.921 | Parallel_RC | 1300.0  | 47.0    |
| 1     | 1179.379  | -27.790 | Parallel_RC | 1300.0  | 47.0    |
| 2     | 1088.752  | -33.583 | Parallel_RC | 1300.0  | 47.0    |
| 3     | 999.412   | -40.792 | Parallel_RC | 1300.0  | 47.0    |

### 2.5.3 Flattening Frequency Sequence Data into Machine Learning Feature Vectors

Each sweep is grouped into a `Sample_ID` using the `Index=0` marker. A pivot operation is applied to create a single data row per sample with a set of columns: `Magnitude_0`, ..., `Magnitude_14` and `Phase_0`, ..., `Phase_14`. The result is saved to `final_data.csv` for training.

- **Before processing:** Data is in a long format. Each EIS sweep consists of 15 measurement points (from `Index 0` to `14`), scattered across 15 data rows.

- **Processing method:** The algorithm iterates through the data and groups points belonging to the same sweep cycle based on the `Index=0` marker, assigning them a common `Sample_ID` [23]. Subsequently, the pivot technique is used to rotate the data axis, merging the 15 data-point rows into a single vector row.
- **After processing:** Data is in a wide format. Each row represents a complete spectrum sweep, containing 34 columns:
  - Identification & labels: `Sample_ID`, `Label`, `Value_R`, `Value_C`
  - Magnitude features: `Magnitude_0` to `Magnitude_14`
  - Phase features: `Phase_0` to `Phase_14`

Table 8: Example rows of the final flattened dataset

| <code>Sample_ID</code> | <code>Label</code> | <code>Value_R</code> | <code>Value_C</code> | <code>Mag_0</code> | ... | <code>Mag_14</code> | <code>Phase_0</code> | ... | <code>Phase_14</code> |
|------------------------|--------------------|----------------------|----------------------|--------------------|-----|---------------------|----------------------|-----|-----------------------|
| 1                      | nothing            | 0.0                  | 0.0                  | 139 435.6          | ... | 177 898.9           | -147.19              | ... | -152.67               |
| 2                      | Parallel_RC        | 1300.0               | 100.0                | 1098.359           | ... | 191.294             | -36.673              | ... | -82.793               |
| 3                      | Resistor           | 2400.0               | 0.0                  | 2466.725           | ... | 2450.957            | 0.828                | ... | 3.667                 |
| 4                      | Capacitor          | 0.0                  | 100.0                | 1716.186           | ... | 172.145             | -88.41               | ... | -90.837               |

### 2.5.4 Dual Scaling Strategy

In this project, we use two scaling layers are utilized in parallel, depending on the task’s objective.

**For input  $X$ .**

- The classification branch uses `MinMaxScaler` [24].
- The new CNN regression branch uses `RobustScaler` for Magnitude and Phase to reduce sensitivity to outliers [24].

**For output  $y$ .**

- The classical regression branch applies a  $\log_{10}$  transformation for `Value_R` and `Value_C`, then reverses it using  $10^y$  during inference.

In terms of reporting, the dual scaling strategy can be presented as follows.

**1. Input amplitude scaling via min–max** (prioritizing local scaling for the embedded task to increase stability across measurement boards) [25]:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} + \epsilon \quad (27)$$

**2. Compressing the dynamic range of outputs  $R$  and  $C$**  [25, 5]:

$$y' = \log_{10}(y + \epsilon), \quad y = 10^{y'} \quad (28)$$

This aligns with the nature of EIS data: inputs can fluctuate based on measurement amplitude, while the target outputs  $R/C$  span multiple orders of magnitude.

### 2.5.5 Data Leakage Prevention

The CNN regression model prevents data leakage by employing `GroupShuffleSplit` based on a group key:

- `Group_ID = Label + Value_R + Value_C`
- `test_size = 0.2, random_state = 42`

This mechanism forces the same component configuration to reside entirely on either the training or the validation/test side, avoiding the memorization of seen component values [24, 26]. This is a critical step to accurately evaluate the generalization capability on new, unseen components.

## 2.6 Machine Learning Model Design

This section presents the overall model architecture adopted for the project: a two-stage cascade that first classifies the circuit topology and then routes each sample to a topology-specific regressor for parameter estimation [25, 5]. It introduces the rationale behind this decoupled design, while detailing the Random Forest classifier and 1D-CNN regressors that implement Stage 1 and Stage 2, respectively.

### 2.6.1 Two-Stage Cascade Architecture

The system is designed as a two-stage cascade:

- **Stage 1: Topology Classifier**  
Predicts the circuit type: `resistor`, `capacitor`, `series RC`, `parallel RC`.
- **Stage 2: Parameter Regressor**  
Routes each sample, based on the topology predicted in Stage 1, to one of four independently trained CNN regressors that predict only the parameter(s) relevant to that topology.

The main reason for this design is we want to clearly decouple the tasks: the classifier answers “*what is the circuit type?*”, whereas the regressor answers “*what are the parameters  $R$  and  $C$  of this circuit?*”. Training a separate regressor per topology, instead of one shared multi-task head, also means each regressor is no longer forced to hedge between topologies with different parameter ranges.

### 2.6.2 Stage 1 — Topology Classifier (Random Forest)

Main configuration [3, 24]:

- `n_estimators = 100`
- `max_depth = 5`
- `random_state = 42`

The classifier operates on 8 selected frequency points (16 features: magnitude and phase at indices 0, 1, 2, 9, 10, 12, 13, 14 of the 15-point spectrum), scaled with a `MinMaxScaler`. Evaluated on a random split 80% for training and 20% for testing (whole topology/R/C combinations withheld, not just individual rows).

Random Forest was selected because it is :

- Stable with high-dimensional tabular data.
- Less sensitive than a single decision tree.
- Easy to control overfitting by limiting the tree depth.

### 2.6.3 Stage 2 — Topology-Specific Parameter Regressors (1D-CNN)

Four independent instances of the following network (PyTorch) are trained, one per topology; each instance only outputs the parameter(s) that exist for its topology [4, 5] (**Value\_R** only for Resistor, **Value\_C** only for Capacitor, both **Value\_R** and **Value\_C** for Series\_RC and Parallel\_RC):

- **Conv block 1:**
  - Conv1d 2->8, kernel\_size=3, padding=1
  - BatchNorm [27]
  - ReLU [28]
  - Dropout1d(0.1) [29]
  - MaxPool1d(2) [4, 5]
- **Conv block 2:**
  - Conv1d 8->16, kernel\_size=3, padding=1
  - BatchNorm, ReLU, Dropout1d(0.1)
- **Dense head:**
  - Flatten -> Linear(112,32) -> ReLU -> Dropout(0.2) -> Linear(32, 1 or 2)

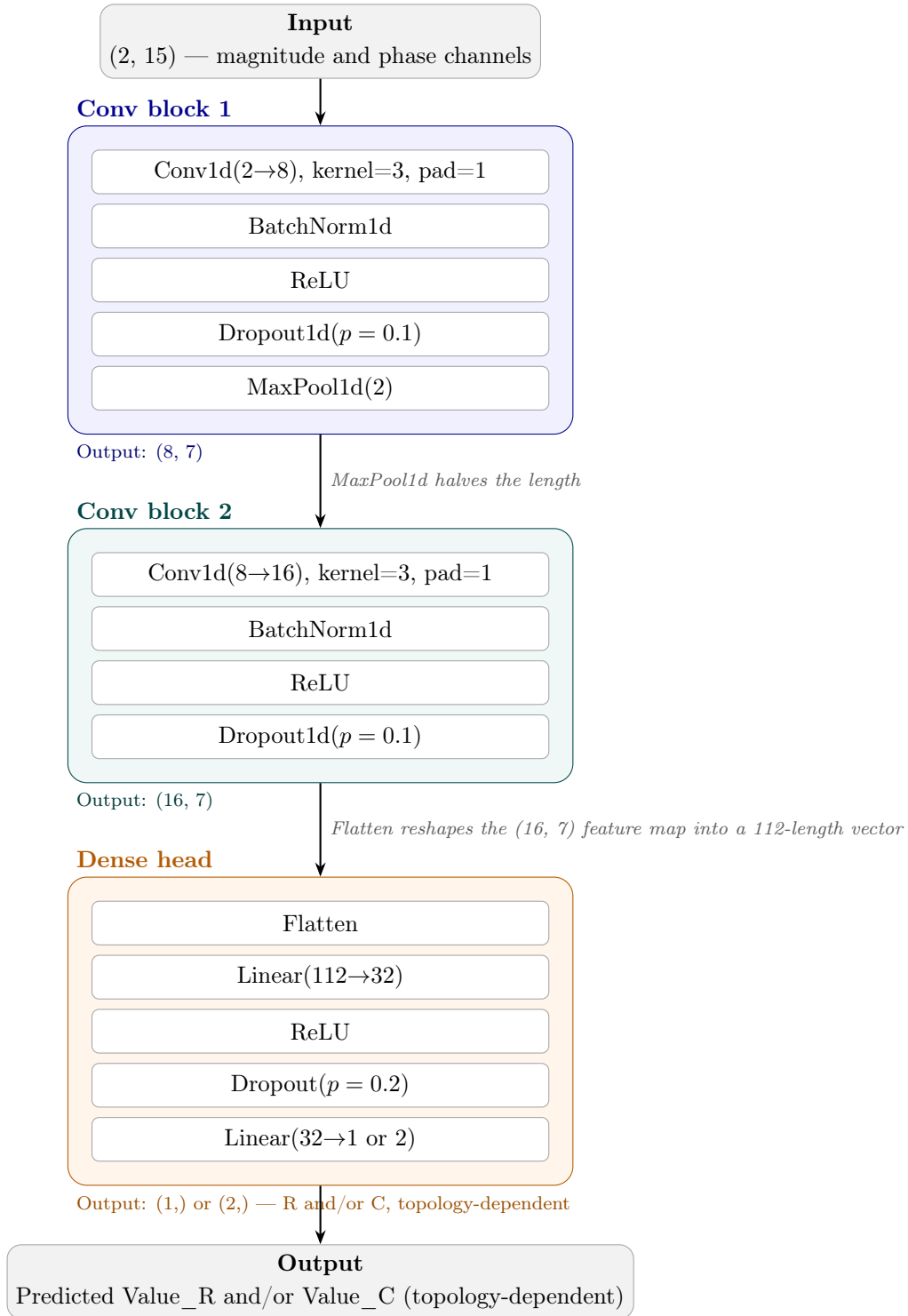


Figure 16: Architecture of the topology-specific 1D-CNN parameter regressor (one independent instance is trained per topology).

## 2.7 Model Training and Deployment

### 2.7.1 Model Training Techniques

The current training pipeline emphasizes robustness and overfitting prevention:

- Splitting datasets using `GroupShuffleSplit` to prevent data leakage.
- Applying independent scalers fitted only on the training set.

- The CNN regressor uses the **AdamW** optimizer, **Huber** loss, a learning-rate scheduler, and early stopping.
- Saving the best model to avoid selecting an overfitted epoch.

Additionally, **Tolerance Injection** can be described as a physical data augmentation strategy:

$$R_{\text{aug}} = R(1 + \delta_R), \quad C_{\text{aug}} = C(1 + \delta_C), \quad \delta \sim \mathcal{U}(-\tau, \tau) \quad (29)$$

The objective is to force the network to learn the response shape rather than memorizing discrete points.

**Repository Status Note.** The tolerance augmentation cell/script is not yet separated at the production level. This can be documented in the report as a module currently being finalized during the training engineering phase.

## 2.7.2 Embedded Deployment

Among the options introduced in Table 4, the custom C code-generation approach (a Python export script converting the trained scikit-learn Random Forest and PyTorch CNN regressors directly into portable C, requiring only `<math.h>` and `<string.h>`) was selected because it is the only option that directly supports the project's two-stage cascade architecture without restructuring the model or maintaining two separate inference pipelines. However, this flexibility requires an explicit trade-off: we could not utilize hardware-optimized kernels provided by the vendor and automatic quantization.

Quantization is not used in this project because the target microcontroller (STM32H7 series) includes a hardware single/double-precision FPU, so it can work with `float32` arithmetic efficiently. Furthermore, the regression stage predicts a value in the  $\log_{10}$  domain, which is then inverse-transformed ( $10^x$ ) to recover  $R$  and  $C$ . This step is sensitive to small absolute errors in the scaled/log domain so full precision `float32` is more suitable in this project.

In conclusion, the procedure of the deployment of the model on target microcontroller will happen as follows: dataset is collected and used to train the model, then related files (model files, scaler files, label encoder files, configuration files, parameter files) will be then converted into one single C header file, which will be included and used in the program code of the target microcontroller.

The trained classification and regression cascade model (Random Forest classifier + per-class 1D-CNN regressors) is deployed on the STM32H755ZI (Nucleo-H755ZI-Q) microcontroller by the custom C code-generation script mentioned in Section 4.10.

The system now operates based on the original workflow (Figure 12) with some adjustments. To be specific, measurement results will not be transmitted to PC anymore but classification and regression results of each measurement iteration will be printed on PC instead. The system utilize the 100-iteration measurement loop (which is originally used for fast dataset collecting) to improve the accuracy of the final result. The final result is obtained via majority vote (classification) and averaging (regression) across the votes matching the winning class.

### 3 Results and Evaluation

This chapter presents a comprehensive evaluation of the developed embedded impedance spectroscopy platform, bridging the hardware realization, digital signal processing, and machine-learning inference stages established in Chapters 1 and 2. Section 3.1 defines the evaluation methodology and metrics used to assess the system. Section 3.2 then investigates the fidelity of the acquired complex impedance spectra against theoretical reference models across the entire excitation frequency band, while Section 3.3 reports the classification and regression performance of the machine-learning cascade. Section 3.4 evaluates the practical resource utilization and worst-case execution time of the deployed models on the target microcontroller. Finally, Section 3.5 concludes the chapter with a critical discussion addressing the inherent limitations observed in both the analog measurement hardware and the data-driven machine-learning approach.

#### 3.1 Evaluation Methodology

The system is evaluated across three independent but interconnected stages:

- **IS Measurement Stage:** Use to verify whether the measured impedance signal accurately reflects the frequency-dependent behavior of the DUT.
- **ML Inference Stage:** Use to correctly predict the component state or component values from the extracted EIS features (Magnitude/Phase).
- **Embedded Deployment Stage:** Use to measure the Flash/RAM footprint and the WCET (worst-case execution time) of ML model on target board.

##### Ground Truth / Reference:

- For the measurement stage: The theoretical reference curve of each R-C configuration across the frequency sweep.
- For the ML classification stage: The encoded class labels (**Label**).
- For the ML regression stage: The continuous target values **Value\_R** and **Value\_C**.

##### Evaluation Metrics:

- **IS Measurement Stage:**
  - **Relative Complex Error (RCE):** Evaluated to quantify the overall complex impedance deviation, combining both magnitude and phase discrepancies at each discrete frequency  $f_i$ :

$$\text{RCE}(f_i) = \frac{|Z_{\text{meas}}(f_i) - Z_{\text{theory}}(f_i)|}{|Z_{\text{theory}}(f_i)|} \times 100\% \quad (30)$$

where  $Z_{\text{meas}}(f_i) = |Z|_{\text{meas}}(f_i) \times e^{j \cdot \theta_{\text{meas}}(f_i)}$  is reconstructed into a complex number from the empirical magnitude and phase data, and  $|\cdot|$  denotes the complex modulus (Euclidean distance on the complex plane).

- **ML Classification:** Accuracy, Precision, Recall, F1-score, Confusion Matrix, and K-fold Mean  $\pm$  Standard Deviation.
- **ML Regression:** Validation Loss (Huber),  $\text{MAE}_R(\Omega)$ ,  $\text{MAE}_C$ , and Early Stopping epoch.

## WCET Evaluation

The runtime of the deployed model is measured using the Cortex-M7's built-in DWT (Data Watchpoint and Trace) cycle counter, which provides cycle-accurate timing resolution (2,08 nanosecond resolution), significantly more precise than tick-based timers such as `HAL_GetTick()` (1 millisecond resolution). For each of the 100 measurements, the cycle count will be sampled immediately before and after the model call, and the difference will be converted to microseconds using the known CPU core clock frequency (`SystemCoreClock`). The maximum value among them will be the WCET.

## 3.2 Impedance Measurement Evaluation

This section evaluates the developed impedance spectroscopy hardware. The system is tested across discrete frequencies  $f_i$  using a set of reference passive components, including pure resistors, pure capacitors, and their series/parallel combinations.

Unlike purely resistive loads, the impedance magnitude  $|Z|$  of reactive components shifts significantly across the stimulation bandwidth, directly impacting the measurement accuracy of the analog front-end. Table 9 extracts the RCE at three anchor frequencies (low, mid, and high band) for a set of representative component combinations. These specific combinations were selected to illustrate the most pronounced frequency-dependent error trends observed within each component class.

Table 9: Relative Complex Error (RCE) at Anchor Frequencies for Representative Components

| Component Class | Value (R, C)          | RCE @ $\sim 1$ kHz [%] | RCE @ $\sim 5$ kHz [%] | RCE @ $\sim 10$ kHz [%] | Trend Observation      |
|-----------------|-----------------------|------------------------|------------------------|-------------------------|------------------------|
| Resistor        | 1300 $\Omega$ , 0 nF  | $1.7 \pm 0.8$          | $1.5 \pm 0.6$          | $1.8 \pm 1.0$           | Stable across band     |
| Capacitor       | 0 $\Omega$ , 10 nF    | $6.5 \pm 4.0$          | $2.5 \pm 1.5$          | $3.0 \pm 1.2$           | Improves at high freq  |
| Series RC       | 2400 $\Omega$ , 22 nF | $13.5 \pm 3.0$         | $6.0 \pm 1.5$          | $3.5 \pm 1.0$           | High error at low freq |
| Parallel RC     | 750 $\Omega$ , 100 nF | $4.0 \pm 1.5$          | $15.0 \pm 4.0$         | $22.0 \pm 5.5$          | Degrades at high freq  |

As demonstrated in Table 9, the measurement fidelity is heavily contingent on the resulting total impedance. For instance, the Series RC configuration exhibits maximum error at 1 kHz where its impedance acts as a near-open circuit. Conversely, the Parallel RC network suffers severe accuracy degradation at 10 kHz as its equivalent impedance approaches a short-circuit condition, pushing the drive circuitry toward its physical limits.

To visualize this physical limitation, Figure 17 illustrates the full RCE distribution for the Parallel RC class. The growth of the error boundary as the frequency approaches 10 kHz clearly visualizes the bandwidth bottleneck of the analog front-end when driving low-impedance reactive loads.



Figure 17: Frequency-dependent RCE distribution for the Parallel RC class. The shaded regions denote the standard deviation ( $\pm 1$  STD)

Due to the extensive volume of the parametric sweeps across various RC combinations, the complete spatial distribution of the RCE over the entire frequency grid for the remaining component classes (Pure Resistor, Pure Capacitor, and Series RC) is compiled and provided in Appendix A.

Table 10: Measurement Repeatability and Dispersion ( $\pm 1$  STD) Across Representative Operating Regimes

| Component Class | Representative State  | STD @ 1 kHz [%] | STD @ 5 kHz [%] | STD @ 10 kHz [%] | Stability Characterization     |
|-----------------|-----------------------|-----------------|-----------------|------------------|--------------------------------|
| Resistor        | 1300 $\Omega$ (Pure)  | $\pm 0.8$       | $\pm 0.6$       | $\pm 1.0$        | Highly stable baseline noise   |
| Capacitor       | 10 nF (Pure)          | $\pm 4.0$       | $\pm 1.5$       | $\pm 1.2$        | Improves at higher frequencies |
| Series RC       | 2400 $\Omega$ , 22 nF | $\pm 3.0$       | $\pm 1.5$       | $\pm 1.0$        | High dispersion at low-band    |
| Parallel RC     | 750 $\Omega$ , 100 nF | $\pm 1.5$       | $\pm 4.0$       | $\pm 5.5$        | Expanded variance at high-band |

To quantitatively substantiate the system's repeatability, Table 10 breaks down the standard deviation ( $\pm 1$  STD) across key frequency milestones. The numerical dispersion reveals two distinct behaviors:

- **Baseline Stability:** Pure resistive configurations maintain tight repeatability ( $\pm 0.6\%$  to  $\pm 1.0\%$ ), confirming minimal internal jitter under steady-state conditions.
- **Impedance-Dependent Variance:** Reactive components exhibit load-sensitive dispersion. Specifically, the Series RC configuration displays wider uncertainty bounds ( $\pm 3.0\%$ ) at 1 kHz due to lowered signal amplitudes, whereas the Parallel RC network broadens to  $\pm 5.5\%$  at 10 kHz as analog front-end saturation effects set in.

### 3.3 Machine Learning Results

#### A. Classification Results

Executed test accuracy results [25, 24]:

Table 11: Detailed classification report for the evaluated Machine Learning models.

| Model (Overall Accuracy)         | Class       | Precision | Recall | F1-score | Support |
|----------------------------------|-------------|-----------|--------|----------|---------|
| <b>Decision Tree</b> (98.46%)    | Capacitor   | 0.87      | 0.96   | 0.91     | 180     |
|                                  | Parallel_RC | 1.00      | 1.00   | 1.00     | 1080    |
|                                  | Resistor    | 0.98      | 0.99   | 0.98     | 180     |
|                                  | Series_RC   | 0.99      | 0.98   | 0.99     | 1260    |
|                                  | nothing     | 1.00      | 1.00   | 1.00     | 30      |
| <b>Random Forest</b> (98.75%)    | Capacitor   | 0.88      | 0.98   | 0.93     | 180     |
|                                  | Parallel_RC | 0.99      | 1.00   | 1.00     | 1080    |
|                                  | Resistor    | 1.00      | 0.96   | 0.98     | 180     |
|                                  | Series_RC   | 1.00      | 0.98   | 0.99     | 1260    |
|                                  | nothing     | 1.00      | 1.00   | 1.00     | 30      |
| <b>SVM (RBF Kernel)</b> (96.56%) | Capacitor   | 0.79      | 0.99   | 0.88     | 180     |
|                                  | Parallel_RC | 0.96      | 1.00   | 0.98     | 1080    |
|                                  | Resistor    | 0.97      | 0.81   | 0.88     | 180     |
|                                  | Series_RC   | 1.00      | 0.96   | 0.98     | 1260    |
|                                  | nothing     | 1.00      | 1.00   | 1.00     | 30      |
| <b>MLP (Perceptron)</b> (98.28%) | Capacitor   | 0.87      | 0.97   | 0.92     | 180     |
|                                  | Parallel_RC | 0.99      | 1.00   | 0.99     | 1080    |
|                                  | Resistor    | 1.00      | 0.91   | 0.95     | 180     |
|                                  | Series_RC   | 1.00      | 0.98   | 0.99     | 1260    |
|                                  | nothing     | 1.00      | 1.00   | 1.00     | 30      |

K-fold cross-validation results [25, 24] (classical models):

Table 12: K-fold cross-validation results and Train-Test gap for classical models.

| Model                | K-fold Mean Accuracy | Standard Deviation | Train-Test Gap |
|----------------------|----------------------|--------------------|----------------|
| Decision Tree        | 97.71%               | $\pm 0.70\%$       | +0.61%         |
| Random Forest        | <b>98.34%</b>        | $\pm$ <b>0.74%</b> | +0.49%         |
| SVM (RBF Kernel)     | 95.08%               | $\pm 0.97\%$       | +0.44%         |
| MLP (Perceptron) [5] | 97.02%               | $\pm 0.77\%$       | + <b>0.29%</b> |

As the result, we can analyze that all models achieved an exceptional baseline accuracy exceeding 95%, demonstrating that the extracted EIS features which are Magnitude and Phase across frequencies provide a highly separable representation of the component topologies.

Random Forest emerged as the most robust model, yielding the highest test accuracy (98.75%) and K-fold mean (98.34%). This highlights the effectiveness of decision-making in high-dimensional spectral data.

SVM with the RBF kernel performed slightly worse because its performance depends strongly on the selected hyperparameters [24, 30], especially  $C$  and  $\gamma$ . The EIS dataset also contains class imbalance, with more Series-RC and Parallel-RC samples than Resistor, Capacitor, and nothing samples. This can make it harder for SVM to construct an optimal boundary for every class.

The gaps are uniformly small across all models, demonstrating strong generalization capabilities on the unseen test data.

## B. Regression Results

Executed validation results for the parameter regressors (Stage 2 of the cascade), on the held-out set of 1710 rows spanning 18 (topology, R, C) combinations never seen during training:

Table 13: Regression performance (Mean Absolute Error) by circuit topology, under oracle routing (ground-truth topology) and end-to-end routing (topology predicted by the Stage-1 classifier).

| Routing Mode                 | Topology    | MAE_R ( $\Omega$ ) | MAE_C (nF) | Support |
|------------------------------|-------------|--------------------|------------|---------|
| <b>Oracle</b> (upper bound)  | Capacitor   | 0.00               | 10.27      | 190     |
|                              | Parallel_RC | 196.80             | 3.66       | 855     |
|                              | Resistor    | 162.03             | 0.00       | 95      |
|                              | Series_RC   | 171.19             | 5.52       | 570     |
| <b>End-to-end</b> (deployed) | Capacitor   | 0.00               | 10.27      | 190     |
|                              | Parallel_RC | 196.80             | 3.66       | 855     |
|                              | Resistor    | 162.03             | 0.00       | 95      |
|                              | Series_RC   | 174.74             | 5.62       | 570     |

Overall regression accuracy and the cost of imperfect topology routing:

Table 14: Overall MAE

| Routing Mode       | Overall MAE_R ( $\Omega$ ) | Overall MAE_C (nF) | Misclassified    |
|--------------------|----------------------------|--------------------|------------------|
| Oracle routing     | 185.03                     | 5.09               | —                |
| End-to-end routing | <b>186.36</b>              | <b>5.13</b>        | 6 / 1710 (0.35%) |
| <b>Routing gap</b> | +1.33                      | +0.04              |                  |

As the result, we can analyze that the Stage-1 classifier’s routing error has almost no measurable impact on the deployed (end-to-end) regression accuracy: only 6 of the 1710 held-out rows (0.35%) were misclassified, all belonging to the same group (**Series\_RC\_360 $\Omega$ \_47nF** routed to the Capacitor regressor), so the overall MAE\_R and MAE\_C rise by only +1.33  $\Omega$  and +0.04 nF over the oracle upper bound. This confirms that decoupling topology classification from parameter regression, as opposed to a single joint multi-task network, does not trade classification error for regression error in practice.

Capacitor’s MAE\_R of 0.00  $\Omega$  and Resistor’s MAE\_C of 0.00 nF are architectural, not learned: each topology-specific regressor only outputs the parameter(s) that exist for that topology (Section 16), so R is never predicted for a pure capacitor and C is never predicted for a pure resistor.

The largest individual-group errors are concentrated at the edges of the component grid and are physically explainable rather than a modeling failure, for example: **Parallel\_RC\_4300 $\Omega$ \_68nF** has the single worst MAE\_R (834.27  $\Omega$ ), because at this corner frequency the parallel combination’s impedance magnitude never approaches the true  $R$  within the measured frequency band, making  $R$  weakly observable from magnitude alone; **Series\_RC\_4300 $\Omega$ \_33nF** shows a similar effect (504.76  $\Omega$ ). Conversely, **Series\_RC\_2400 $\Omega$ \_100nF** has the worst MAE\_C (22.37 nF), since  $1/(\omega C) \ll R$  across most of the band, so the response is dominated by  $R$  and  $C$  becomes only weakly identifiable. These cases are reported here as known, physically-grounded limitations of the measured frequency band rather than deficiencies to be tuned away.

### 3.4 Embedded Deployment Results

|                               |                                       |
|-------------------------------|---------------------------------------|
| <b>Flash (Before - After)</b> | 80,29 KB (3,92%) - 358,02 KB (17,48%) |
| <b>RAM (Before - After)</b>   | 35,54 KB (3,47%) - 35,63 KB (3,48%)   |
| <b>WCET</b>                   | 13057,48 $\mu$ s                      |

Table 15: Hardware resource utilization and WCE=0T on the STM32H755ZI.

Table 15 summarizes the memory and timing overhead introduced by deploying the Random Forest + CNN cascade model on the STM32H755ZI (Cortex-M7 core). The usage percentage is calculated with respect to the total memory of the target microcontroller (2 MB Flash memory, 1 MB RAM). Flash usage increased from 80,29 KB (3,92%) to 358,02 KB (17,48%) of the 2 MB Flash, an increase of roughly 277.7 KB attributable to the model’s stored weights and decision-tree branching code. RAM usage remained largely unaffected, rising only marginally from 35,54 KB (3,47%) to 35,63 KB (3,48%) of 1 MB RAM, since the model’s static footprint (vote counters and accumulators) is negligible; the bulk of its working memory is instead allocated transiently on the stack during inference. The measured worst-case execution time (WCET) for a single call was 13057,48  $\mu$ s, which is acceptable for this project.

### 3.5 Limitations

#### 3.5.1 IS Measurement System Limitations

Despite the overall validation of the impedance spectroscopy (IS) hardware across the tested frequency spectrum, several physical and methodological constraints impact the system’s accuracy and performance boundaries:

- **Prototyping Hardware Architecture:** The hardware is currently implemented on a standard soldered protoboard (perfboard) rather than a dedicated multi-layer printed circuit board (PCB). This introduces unoptimized parasitic capacitances, stray inductances, and higher contact resistances, which distort high-frequency signals.
- **Electromagnetic Interference (EMI) Susceptibility:** Due to the lack of dedicated ground planes and electromagnetic shielding on the prototype board, the analog front-end is sensitive to ambient electromagnetic interference and power supply ripple, significantly degrading the quality of measurement.
- **Nominal Component Value Baseline:** The theoretical impedance  $Z_{\text{theory}}$  is computed using the manufacturer’s nominal component values rather than precise measurements from a calibrated industrial LCR meter. Consequently, the evaluated Relative Complex Error (RCE) inherently absorbs the component manufacturing tolerances (e.g.,  $\pm 5\%$  to  $\pm 20\%$  variations).

#### 3.5.2 Machine Learning Approach

Although the classification and regression results are promising, several practical limitations regarding the dataset and model architecture must be considered:

- **Controlled Experimental Conditions:** The dataset consists of a limited set of component values collected under ideal conditions. Performance may degrade in real-world deployments due to out-of-range values, component ageing, temperature drifts, parasitic inductance, or varying measurement setups.

- **Class Imbalance:** The dataset is heavily skewed towards Series-RC and Parallel-RC topologies compared to the Resistor, Capacitor, and **nothing** classes [31]. This limits the model’s ability to learn natural variations of the minority classes, potentially reducing reliability for rare circuit conditions.
- **Aggressive Outlier Removal:** The Isolation Forest algorithm was used to remove 5% of the data [32]. While this improves dataset compactness, it may discard physically valid anomalies caused by connection quality or component tolerances, leading to over-optimistic performance metrics.
- **Data Splitting Strategy:** The classification stage employs a stratified random split, which may allow similar impedance patterns (identical  $R$  and  $C$  combinations) in both training and validation sets. Although the regression stage mitigates this using **Group ShuffleSplit**, relying on a single random split (**SEED** = 42) limits the reliability of the Mean Absolute Error (MAE) estimate. Repeated group cross-validation is recommended for stricter evaluation.
- **Cascade Structure Error Propagation:** The system relies on a Random Forest classifier to route inputs to topology-specific CNN regressors. Misclassifications inevitably lead to severe estimation errors. Consequently, end-to-end performance may be lower than the oracle-routing evaluation, where the true topology is known.
- **Logarithmic Error Scaling:** The CNN regressors predict the logarithm of component values using **RobustScaler**. Since small errors in the logarithmic domain translate to large absolute errors for high component values, MAE alone is insufficient [25]. The model should also be evaluated using relative error:

$$\text{Relative Error} = \frac{|\hat{y} - y|}{y} \times 100\% \quad (31)$$

- **Closed-World Assumption:** The current pipeline can only predict topologies present in the training set. Introducing an unknown circuit will result in forced misclassification [33].

These limitations do not invalidate the findings but define the operational boundaries of the current setup and highlight key directions for improving real-world robustness.

## 4 Conclusion

### 4.1 Summary

This project set out to determine whether a complete impedance spectroscopy chain – from excitation to equivalent-circuit parameter estimation – can be realised on a single general-purpose microcontroller. The resulting system combines three subsystems: an analog front-end and firmware that acquire the spectrum, an on-device inference cascade that interprets it, and a host application that visualises the measurement and builds the training dataset.

On the measurement side, the I–V method was adopted because it interfaces directly with the DAC and ADC peripherals already integrated in the STM32, avoiding any analog balancing network. A crest-factor-optimised multisine excitation replaces the sequential sine sweep, so that all frequencies of interest are probed within a single short acquisition window, which keeps the total measurement time compatible with the time-invariance requirement. Spectral leakage is avoided by deriving the DAC output and both ADC channels from a common timer, and by placing every excited tone on an exact frequency bin of the analysis window. DMA-driven transfers keep the CPU free during acquisition, and the frequency-domain analysis stage delivers

the magnitude and phase of the voltage and current at each excited frequency, from which the complex impedance of the DUT is obtained.

On the inference side, the classical offline curve-fitting step was replaced by a two-stage cascade. A Random Forest first classifies the circuit topology, and one of four independently trained 1D convolutional neural networks then estimates only the parameters that physically exist for that topology. Decoupling the two tasks means that no single regressor has to hedge between topologies with completely different parameter ranges, and it keeps each model small enough for the target. Because none of the available vendor frameworks supports this hybrid structure natively, the trained models, scalars and label encoder were exported into a single dependency-free C header file by a custom code-generation script.

## 4.2 Main Results

The evaluation confirms that the concept is viable, while also showing clearly where its present boundaries lie:

- **Impedance measurement.** For purely resistive loads the relative complex error stays below roughly 2 % across the whole excitation band. For reactive networks the error becomes strongly frequency-dependent: series RC combinations are least accurate at low frequencies, where the network approaches an open circuit, whereas parallel RC combinations degrade towards the upper end of the band, where the equivalent impedance approaches a short circuit and the drive circuitry reaches its limits.
- **Topology classification.** The Random Forest reached 98.75 % test accuracy with a K-fold mean of 98.34 %, outperforming the Decision Tree, SVM (RBF) and MLP baselines. The consistently small train–test gap of all evaluated models indicates that the magnitude and phase features form a well-separable representation of the four topologies.
- **Parameter regression.** Under group-based validation, i.e. on component combinations that were entirely withheld from training, the cascade achieved an overall mean absolute error of  $186.36\ \Omega$  for  $R$  and  $5.13\ \text{nF}$  for  $C$ . Only 0.35 % of the held-out samples were routed to the wrong regressor, which raised the error by no more than  $1.33\ \Omega$  and  $0.04\ \text{nF}$  with respect to oracle routing. Error propagation through the cascade is therefore negligible in practice. The largest individual errors occur at the edges of the component grid and are physically explainable: they correspond to configurations in which one of the two parameters is only weakly observable within the measured frequency band.
- **Embedded deployment.** Adding the complete cascade increased flash usage from 3.92 % to 17.48 % of the 2 MB available, while RAM usage remained essentially unchanged at about 3.5 % of 1 MB, since the working memory of the inference is allocated transiently on the stack. The measured worst-case execution time of a single inference call was approximately 13.06 ms, which is far below the duration of one acquisition cycle and therefore does not limit the measurement rate.

Taken together, these figures show that the microcontroller is not the bottleneck of the system. The dominant source of uncertainty is the analog front-end, in particular when driving low-impedance reactive loads near the upper end of the excitation band.

## 4.3 Limitations

The results must be read within the operating conditions under which they were obtained. The analog front-end is built on a soldered protoboard without a dedicated ground plane or shielding, which introduces parasitic elements and sensitivity to electromagnetic interference. The reference impedance used to compute the relative complex error is based on nominal component values

rather than on measurements from a calibrated LCR meter, so the reported error also absorbs component manufacturing tolerances. On the machine-learning side, the dataset was recorded under controlled laboratory conditions, is imbalanced towards the series and parallel RC classes, and covers a discrete grid of component values. Finally, the pipeline operates under a closed-world assumption: any DUT that does not belong to one of the four trained topologies will necessarily be forced into one of them. A detailed discussion of these points is given in Section 3.5.

In summary, the project demonstrates a complete, self-contained path from multisine excitation to on-device equivalent-circuit parameter estimation on a low-cost microcontroller platform. The measurement accuracy is presently limited by the prototype analog front-end rather than by the digital or the machine-learning stage, which indicates that the concept scales well and that the most valuable improvements lie in hardware refinement and in broadening the range of devices the system has been trained to interpret.

## 5 References

### References

- [1] O. Kanoun and A. Kallel, “High-performance efficient embedded systems for impedance spectroscopy: Challenges and potentials,” *Electrochimica Acta*, vol. 492, p. 144351, 04 2024.
- [2] STMicroelectronics, *AN4841 Application Note: Digital signal processing for STM32 microcontrollers using CMSIS*. STMicroelectronics, 2021. Rev 3.
- [3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [6] A. Y. Kallel and O. Kanoun, “Crest factor optimization for multisine excitation signals with logarithmic frequency distribution based on a hybrid stochastic-deterministic optimization algorithm,” *Batteries*, vol. 8, no. 10, 2022.
- [7] Analog Devices, Inc., *AD8421: 3 nV/Hz, Low Power Instrumentation Amplifier Data Sheet*, 2012. Rev. 0.
- [8] Analog Devices, Inc., *ADA4522-1/ADA4522-2/ADA4522-4: 55 V, EMI Enhanced, Zero Drift, Ultralow Noise, Rail-to-Rail Output Operational Amplifiers*, 2015. Rev. I.
- [9] STMicroelectronics, “Nucleo-h755zi-q: Stm32 nucleo-144 development board with stm32h755zi mcu.”
- [10] M. F. Kilkenny and K. M. Robinson, “Data quality: “garbage in – garbage out”,” *Health Information Management Journal*, vol. 47, no. 3, pp. 103–105, 2018.
- [11] R. S. Geiger, D. Cope, J. Ip, M. Lotosh, A. Shah, J. Weng, and R. Tang, ““garbage in, garbage out” revisited: What do machine learning application papers report about human-labeled training data?,” *Quantitative Science Studies*, vol. 2, no. 3, pp. 795–827, 2021.
- [12] E. Barsoukov and J. R. Macdonald, *Impedance Spectroscopy: Theory, Experiment, and Applications*. Hoboken, NJ, USA: John Wiley & Sons, 3 ed., 2018.

- [13] JCGM, *International Vocabulary of Metrology — Basic and General Concepts and Associated Terms (VIM)*, 3rd edition. Joint Committee for Guides in Metrology, 2012.
- [14] C. Liechti, “pySerial documentation, release 3.5.” <https://pyserial.readthedocs.io/>, 2020. Accessed: 2026-07-23.
- [15] The Qt Company, *Qt Documentation*, 2024. Accessed: 2026-07-23.
- [16] A. Lasia, *Electrochemical Impedance Spectroscopy and its Applications*. New York, NY, USA: Springer, 2014.
- [17] M. E. Orazem and B. Tribollet, *Electrochemical Impedance Spectroscopy*. The ECS Series of Texts and Monographs, Hoboken, NJ, USA: John Wiley & Sons, 2 ed., 2017.
- [18] B. A. Boukamp, “A linear Kronig-Kramers transform test for immittance data validation,” *Journal of the Electrochemical Society*, vol. 142, no. 6, pp. 1885–1894, 1995.
- [19] S. Grimnes and Ø. G. Martinsen, *Bioimpedance and Bioelectricity Basics*. Oxford, UK: Academic Press (Elsevier), 3 ed., 2015.
- [20] J. D. Muñoz, V. H. Mosquera, and C. F. Rengifo, “A low-cost, portable, two-dimensional bioimpedance distribution estimation system based on the AD5933 impedance converter,” *HardwareX*, vol. 11, p. e00274, 2022.
- [21] J. Ferreira, F. Seoane, and K. Lindecrantz, “AD5933-based spectrometer for electrical bioimpedance applications,” *Journal of Physics: Conference Series*, vol. 224, p. 012011, 2010.
- [22] W. McKinney, “Data structures for statistical computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (S. van der Walt and J. Millman, eds.), (Austin, TX, USA), pp. 56–61, 2010.
- [23] The pandas development team, “pandas: Powerful Python data analysis toolkit — reshaping and pivot tables.” [https://pandas.pydata.org/docs/user\\_guide/reshaping.html](https://pandas.pydata.org/docs/user_guide/reshaping.html), 2023.
- [24] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [25] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. New York: Springer, 2nd ed., 2009.
- [26] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, 2012.
- [27] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.
- [28] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pp. 807–814, 2010.
- [29] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

- [30] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [31] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [32] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422, 2008.
- [33] W. J. Scheirer, A. Rocha, A. Sapkota, and T. E. Boult, “Toward open set recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013.

## A Complete RCE Distribution Visualizations

This appendix provides the comprehensive spatial distributions of the Relative Complex Error (RCE) evaluated across all discrete frequency points and component configurations. For each configuration, the solid purple line represents the mean RCE ( $\mu$ ), while the shaded region indicates the standard deviation ( $\pm 1$  STD) derived from repeated measurement trials.

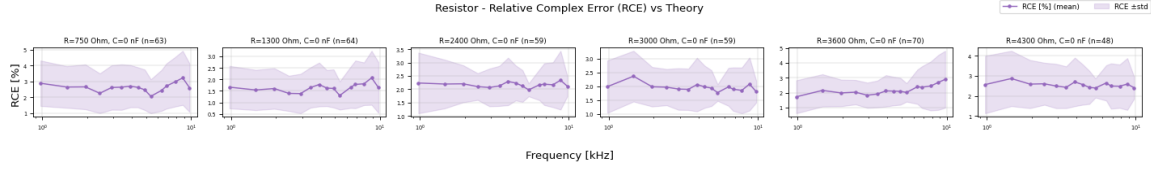


Figure 18: Relative Complex Error (RCE) vs. Theory for Pure Resistor configurations across various resistance values.

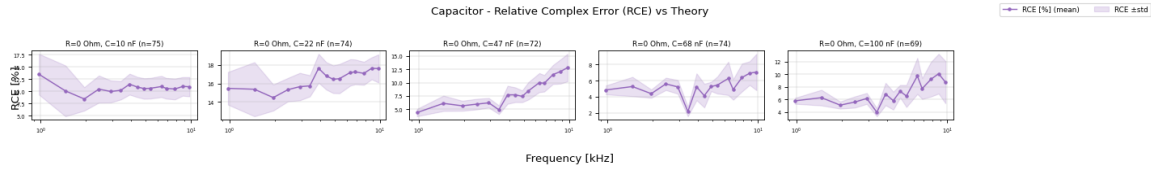


Figure 19: Relative Complex Error (RCE) vs. Theory for Pure Capacitor configurations across various capacitance values.

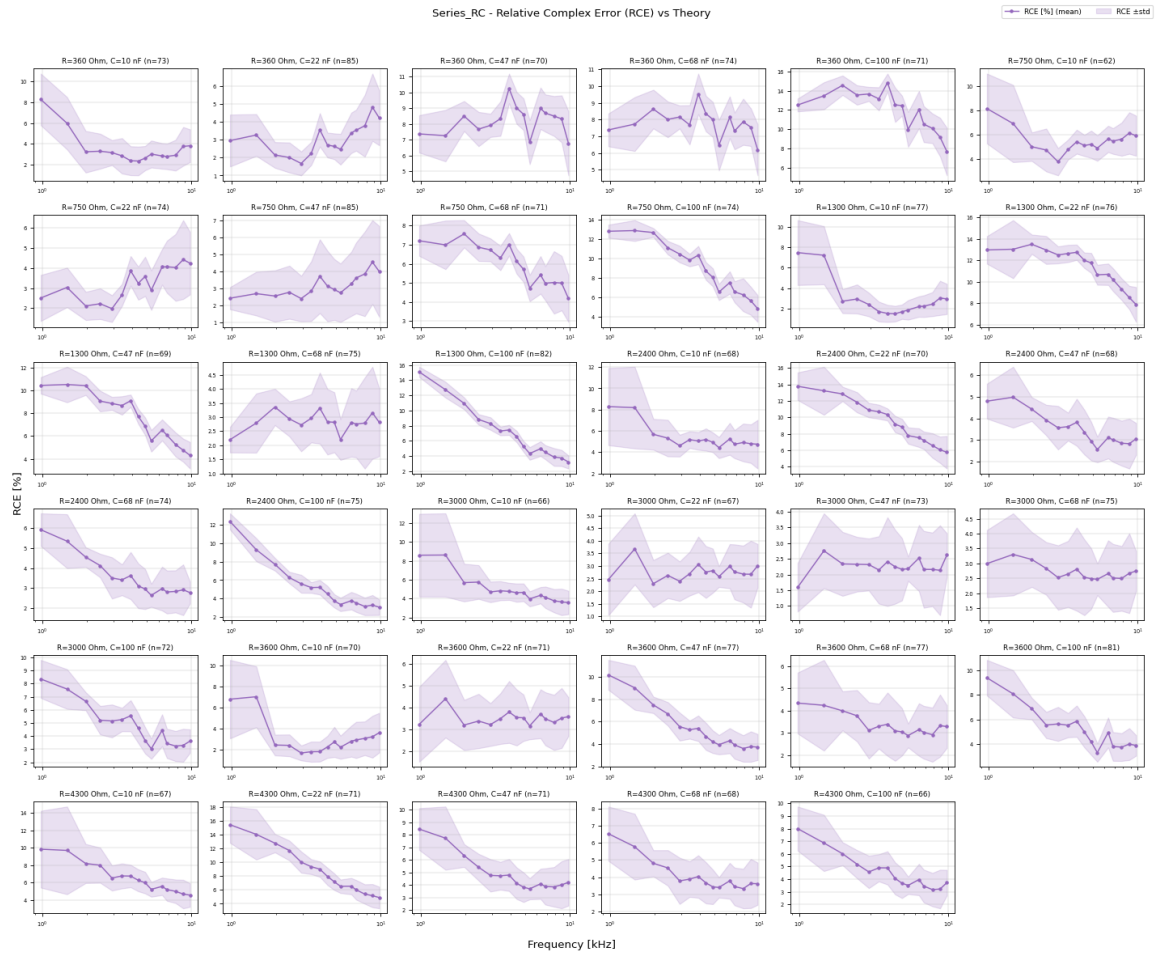


Figure 20: Relative Complex Error (RCE) vs. Theory for Series RC combinations across the tested parameter grid.

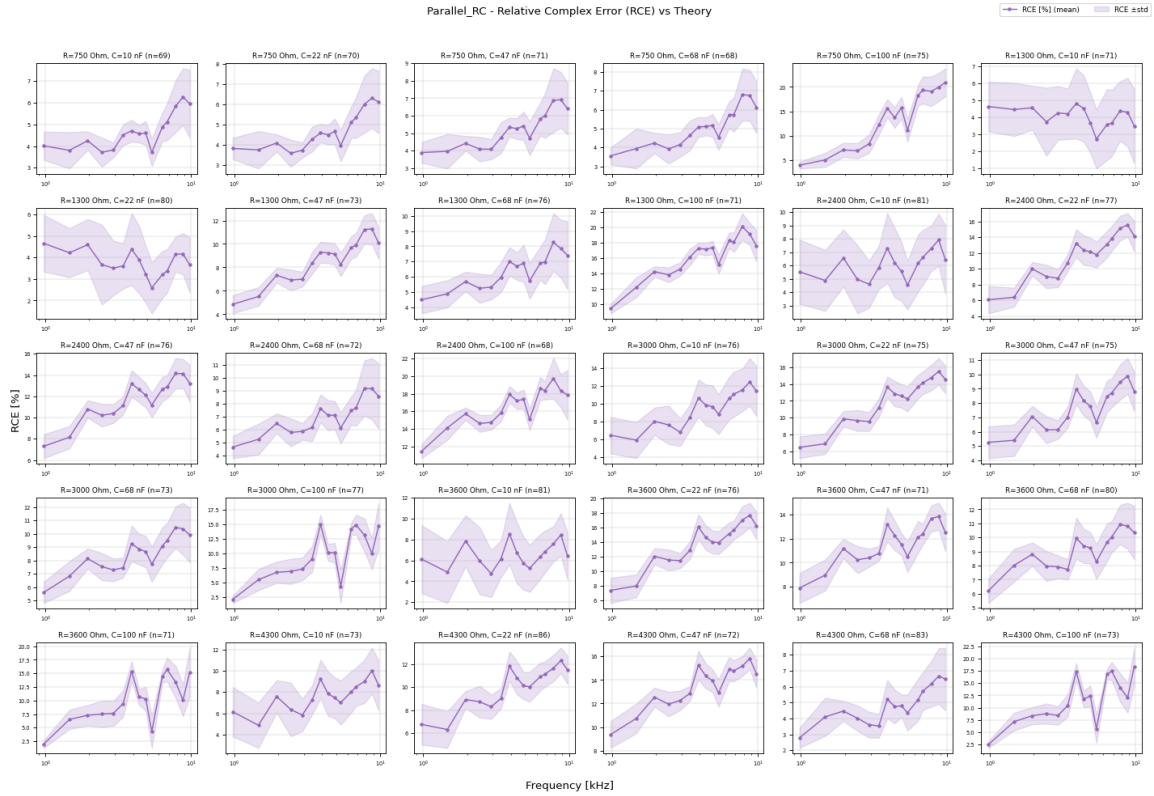


Figure 21: Relative Complex Error (RCE) vs. Theory for Parallel RC combinations across the tested parameter grid.

## B Annex

### Purpose of the Data Pipeline

This annex documents the Python-based data pipeline developed for the machine-learning part of the project. The pipeline transforms recorded impedance spectra into a clean labelled dataset, classifies the circuit topology, and estimates the corresponding resistor and capacitor values.

Each measurement contains impedance magnitude and phase values at 15 selected frequency points. The pipeline consists of the following stages:

1. collection and storage of labelled impedance measurements;
2. removal of invalid and outlying measurements;
3. construction of magnitude and phase feature vectors;
4. topology classification using a Random Forest classifier;
5. component-value regression using topology-specific CNN models;
6. export of validation predictions and error metrics.

### Dataset Format

The cleaned dataset is stored as a CSV file. Each row represents one impedance-spectrum measurement. The relevant columns are listed in Table 16.

The topology labels used for the cascade model are: **Resistor**, **Capacitor**, **Series\_RC**, and **Parallel\_RC**.

Table 16: Main columns used by the machine-learning pipeline.

| Column group               | Description                       |
|----------------------------|-----------------------------------|
| Sample_ID                  | Identifier of the recorded sample |
| Label                      | Circuit topology label            |
| Value_R                    | Resistor value in $\Omega$        |
| Value_C                    | Capacitor value in F              |
| Magnitude_0 – Magnitude_14 | Magnitude at 15 frequency points  |
| Phase_0 – Phase_14         | Phase at 15 frequency points      |

## Data Cleaning and Filtering

Before training, the dataset is filtered to retain only valid component measurements. Samples labelled **nothing** are removed, as are samples where both resistor and capacitor values are zero.

```
1d_cnn_cascade.ipynb -- data filtering cell
```

Listing 1: Filtering valid samples before model training.

```
1 df = pd.read_csv(DATA_PATH)
2
3 df = df[
4     (df['Label'] != 'nothing') &
5     ((df['Value_R'] > 0) | (df['Value_C'] > 0))
6 ].copy()
```

Outlier removal is performed earlier, when generating the final cleaned dataset. The cascade notebook consumes the cleaned file `final_data_outliers_clean_Iso.csv`, ensuring training is not affected by corrupted or physically invalid spectral measurements.

For regression validation, a group identifier is created from the topology and component values so repeated measurements of the same (Label,  $R$ ,  $C$ ) combination cannot be split across training and validation:

```
1d_cnn_cascade.ipynb -- group ID creation cell
```

Listing 2: Creation of groups for regression validation.

```
1 df['Group_ID'] = (
2     df['Label'] + '_' +
3     df['Value_R'].astype(str) + '_' +
4     df['Value_C'].astype(str)
5 )
```

## Feature Preparation

Each sample contains 15 magnitude values and 15 phase values. For CNN regression, the input is arranged as a two-channel tensor:

$$\mathbf{X} \in \mathbb{R}^{2 \times 15}, \quad (32)$$

where the first channel holds impedance magnitude and the second holds phase.

Magnitude and target values are converted to a logarithmic scale before normalization:

$$M_{\log} = \log_{10}(M), \quad y_{\log} = \log_{10}(y). \quad (33)$$

Robust scaling is applied because it uses the median and interquartile range, making it less sensitive to remaining outliers:

$$x_{\text{scaled}} = \frac{x - \text{median}(x)}{\text{IQR}(x)}. \quad (34)$$

```
1d_cnn_cascade.ipynb -- feature preparation cell
```

Listing 3: Preparation of two-channel CNN input data.

```
1 def make_X(df_sub, scaler_mag, scaler_phase, mag_cols, phase_cols):
2     Xm = scaler_mag.transform(np.log10(df_sub[mag_cols]))
3     Xp = scaler_phase.transform(df_sub[phase_cols])
4
5     return np.concatenate(
6         [Xm[:, None, :], Xp[:, None, :]],
7         axis=1
8     )
```

## Cascade Architecture

The model uses a two-stage cascade: a classifier first predicts the circuit topology, then the predicted topology selects the appropriate CNN regressor.

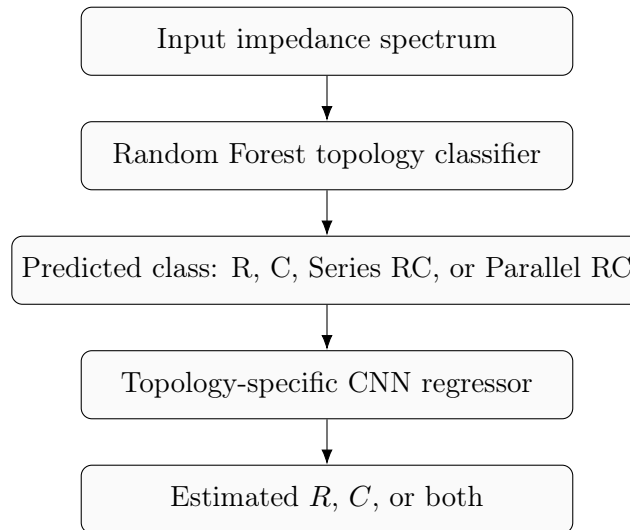


Figure 22: Two-stage cascade: topology classification followed by topology-specific regression.

The target variables depend on the predicted topology, as shown in Table 17.

Table 17: Regression outputs for each topology.

| Topology    | Regression target |
|-------------|-------------------|
| Resistor    | $R$               |
| Capacitor   | $C$               |
| Series RC   | $R$ and $C$       |
| Parallel RC | $R$ and $C$       |

## Random Forest Topology Classifier

The classifier uses magnitude and phase values from eight selected frequency indices {0, 1, 2, 9, 10, 12, 13, 14}, giving 16 input features. Min-max scaling is applied before fitting a Random Forest with 100 trees and a maximum depth of five.

1d\_cnn\_cascade.ipynb -- Random Forest classifier cell

Listing 4: Training of the topology classifier.

```

1 clf_train_df, clf_val_df = train_test_split(
2     df,
3     test_size=0.3,
4     stratify=df['Label'],
5     random_state=SEED
6 )
7
8 le = LabelEncoder().fit(clf_train_df['Label'])
9 clf_scaler = MinMaxScaler().fit(clf_train_df[clf_cols])
10
11 clf = RandomForestClassifier(
12     n_estimators=100,
13     max_depth=5,
14     random_state=SEED
15 )
16
17 clf.fit(
18     clf_scaler.transform(clf_train_df[clf_cols]),
19     le.transform(clf_train_df['Label'])
20 )

```

Classification performance is evaluated using accuracy and a confusion matrix.

## CNN Regression Models

A separate CNN is trained for each topology, allowing it to learn the relationship between spectral shape and component values within one known circuit class. The network uses two 1-D convolutional layers; batch normalization improves training stability, ReLU introduces non-linearity, dropout reduces overfitting, and max pooling reduces the feature length.

1d\_cnn\_cascade.ipynb -- CNN model definition cell

Listing 5: CNN architecture for component-value regression.

```

1 class CNNRegressor(nn.Module):
2     def __init__(self, num_outputs):
3         super().__init__()
4
5         self.backbone = nn.Sequential(
6             nn.Conv1d(2, 8, kernel_size=3, padding=1),
7             nn.BatchNorm1d(8),
8             nn.ReLU(),
9             nn.Dropout1d(0.1),
10            nn.MaxPool1d(2),
11
12            nn.Conv1d(8, 16, kernel_size=3, padding=1),
13            nn.BatchNorm1d(16),
14            nn.ReLU(),
15            nn.Dropout1d(0.1),

```

```

16         nn.Flatten()
17     )
18
19     self.head = nn.Sequential(
20         nn.Linear(16 * 7, 32),
21         nn.ReLU(),
22         nn.Dropout(0.2),
23         nn.Linear(32, num_outputs)
24     )
25
26     def forward(self, x):
27         return self.head(self.backbone(x))

```

The models are trained with AdamW and Huber loss, chosen because it is less sensitive to large regression errors than mean squared error:

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2, & \text{if } |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases} \quad (35)$$

Learning-rate reduction and early stopping avoid unnecessary training once validation performance stops improving.

## Validation Strategy

The regression stage uses `GroupShuffleSplit` with 20% of groups reserved for validation, which tests whether the CNN can estimate component values unseen during training — a stricter test than a random split, where repeated measurements of the same values could appear in both subsets.

```
1d_cnn_cascade.ipynb -- GroupShuffleSplit cell
```

Listing 6: Group-based train-validation split for regression.

```

1 splitter = GroupShuffleSplit(
2     n_splits=1,
3     test_size=0.2,
4     random_state=SEED
5 )
6
7 train_idx, val_idx = next(
8     splitter.split(df, groups=df['Group_ID'])
9 )
10
11 train_df = df.iloc[train_idx].copy()
12 val_df = df.iloc[val_idx].copy()

```

Two regression modes are evaluated:

- **Oracle routing:** the true topology label selects the CNN regressor, measuring regression quality when the topology is known.
- **End-to-end routing:** the label predicted by the Random Forest selects the CNN regressor, representing practical cascade behaviour.

The main regression metric is mean absolute error:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (36)$$

## Output Files

The cascade code exports results to `validation_predictions_cascade.xlsx`, containing:

- classifier accuracy and confusion matrix;
- oracle-routing MAE grouped by topology;
- oracle-routing MAE grouped by component values;
- end-to-end MAE grouped by topology;
- end-to-end MAE grouped by component values;
- individual oracle-routing predictions;
- individual end-to-end predictions.

The final report inserts the numerical classifier accuracy, MAE values, and confusion matrix from the exported workbook after the final experiment is completed.